

Universidad de Guanajuato

Campus Irapuato-Salamanca

División de Ingeniería



Programación en Ingeniería

Reporte de Práctica 1

Cálculo de Series Mediante Estructuras de Control en C

Profesor: Dr. Mario Alberto Ibarra Manzano

Alumnos:

Azuara Yáñez Cecil Quimey

León Amaya Juan Pablo

Rojas Mata Diego Moisés

Salamanca, Guanajuato; 20 de abril de 2026

Índice

1. Objetivo y Metodología.....	3
2. Resultados Numéricos y Evidencias.....	5
3. Análisis de Exactitud y Limitaciones Numéricas.....	32
4. Discusión de Convergencia y Estabilidad	34
5. Arquitectura del Software y Diagramas.....	38
6. Conclusiones y Buenas Prácticas	72
7. Bibliografía.....	73

1. Objetivo y Metodología

El objetivo de la práctica fue implementar un conjunto de 33 series matemáticas en lenguaje de programación C, con el propósito de analizar su convergencia y precisión numérica. Para ello, se evaluó el comportamiento de las aproximaciones al incrementar el número de términos, empleando métricas como la diferencia entre aproximaciones consecutivas (Δ_n) y el error relativo.

El sistema se construyó sobre la siguiente arquitectura de directorios modular:

```
series-approximation-c/
├── .idea/                # Configuración del entorno de desarrollo (CLion).
├── cmake-build-debug/    # Archivos y binarios de compilación generados por CMake.
├── .gitignore
├── CMakeLists.txt        # Configuración principal de compilación para el proyecto.
├── LICENSE
├── README.md
├── Nivel1_Inicial/       # NIVEL 1: SOLUCIÓN DIRECTA.
│   └── main_nivel1.c     # Un solo archivo con las 33 series implementadas de forma simple,  $O(n^2)$ .
├── Nivel2_Intermedio/    # NIVEL 2: ESTABILIZACIÓN Y VALIDACIÓN.
│   ├── include/
│   │   └── series_v2.h   # Declaración de las 33 series y macros de validación.
│   ├── src/
│   │   ├── constantes.c  # Series 1-6 y 27-33.
│   │   ├── exponenciales.c # Series 7-13.
│   │   └── trigonometricas.c # Series 19-26.
│   └── main_nivel2.c     # Validación de dominio. Genera tablas con  $n=\{2,4,8,16,32,64,128,256\}$ .
├── Nivel3_Avanzado/      # NIVEL 3: OPTIMIZACIÓN Y DOCUMENTACIÓN.
│   ├── html/             # Documentación web interactiva generada automáticamente por Doxygen.
│   ├── include/
│   │   └── series_v3.h   # Headers documentados con estándares Doxygen.
│   ├── latex/            # Documentación en formato PDF/LaTeX generada por Doxygen.
│   ├── src/
│   │   ├── especial_opt.c # Implementación optimizada de funciones especiales.
│   │   └── trig_exp_opt.c # Funciones trigonométricas y exponenciales por recurrencia  $O(n)$ .
│   └── main_nivel3.c     # Cálculo de métricas:  $\Delta_n$  (diferencia) y Error de Aproximación.
```

Además, se buscó validar los dominios de convergencia de cada serie, así como comparar el desempeño numérico entre los tipos de dato float y double, identificando limitaciones como pérdida de precisión, overflow y acumulación de error.

El desarrollo del proyecto se estructuró bajo una metodología de tres niveles, con el objetivo de mejorar progresivamente la eficiencia computacional y la estabilidad numérica:

- **Nivel 1** (Inicial – Fuerza bruta, $O(n^2)$): Se implementaron las series de manera directa, recalculando en cada término potencias y factoriales desde cero mediante ciclos internos.
- **Nivel 2** (Intermedio – Producto incremental): Se optimizó el cálculo de cada término utilizando productos acumulativos, evitando el uso explícito de factoriales grandes. Esto reduce el riesgo de overflow y mejora la estabilidad numérica, aunque aún mantiene complejidad $O(n^2)$.

- **Nivel 3** (Avanzado – Recurrencia, $O(n)$): Se implementaron relaciones de recurrencia entre términos consecutivos, reutilizando resultados previos. Este enfoque elimina cálculos redundantes, logrando mayor eficiencia y mejor comportamiento numérico.

También se hizo un uso intencional y balanceado de las estructuras de control en C:

- **do-while:** utilizado principalmente para la validación de entradas del usuario (por ejemplo, asegurar que $n \geq 1$ o que x pertenezca al dominio).
- **for:** empleado para la iteración principal sobre el número de términos de la serie.
- **while:** utilizado en cálculos internos, como la construcción de productos incrementales o evaluaciones auxiliares.

Este enfoque permitió mantener claridad en el código, así como una adecuada distribución de responsabilidades entre los distintos tipos de ciclos.

Finalmente, el código fue documentado utilizando Doxygen, y se desarrollaron diagramas de flujo para representar la lógica de los algoritmos implementados.

2. Resultados Numéricos y Evidencias

A continuación, se presentan las tablas de resultados obtenidas mediante la ejecución del programa main_nivel2.c. Se define la nomenclatura de las columnas: fototeta.c fotopussy.exe

- n (**Octavas de iteración**): Representa el número de términos evaluados en la sumatoria para dicha fila. Se incrementa en potencias de 2 para observar la velocidad de convergencia.
- $S_n(x)$ (**Suma Parcial**): Indica el valor acumulado al que llega la serie matemática tras ejecutar las n iteraciones correspondientes.
- $\Delta_n(x)$ (**Diferencia Absoluta**): Cuantifica la diferencia de valor entre la iteración actual y la anterior.
- $e_a(x)$ (**Error de Aproximación Relativo**): Expresa en notación científica la proporción de incertidumbre del cálculo respecto a su valor actual.

i. Tabla de resultados para aproximación de $\ln(2)$:

n	$S_n(x)$	$\Delta_n(x)$	$e_a(x)$
2	0.500000	5.000000e-01	1.000000e+00
4	0.583333	8.333333e-02	1.428571e-01
8	0.634524	5.119048e-02	8.067542e-02
16	0.662872	2.834804e-02	4.276549e-02
32	0.677766	1.489435e-02	2.197565e-02
64	0.685396	7.629506e-03	1.113153e-02
128	0.689256	3.860481e-03	5.600937e-03
256	0.691198	1.941681e-03	2.809154e-03

ii. Tabla de resultados para aproximación de $\frac{\pi}{4}$:

n	$S_n(x)$	$\Delta_n(x)$	$e_a(x)$
2	0.666667	3.333333e-01	5.000000e-01
4	0.723810	5.714286e-02	7.894737e-02
8	0.754268	3.045843e-02	4.038145e-02
16	0.769788	1.552039e-02	2.016190e-02
32	0.777588	7.799220e-03	1.003002e-02
64	0.781492	3.904583e-03	4.996318e-03
128	0.783445	1.952916e-03	2.492729e-03
256	0.784422	9.765364e-04	1.244913e-03

iii. Tabla de resultados para aproximación de $\frac{\pi^2}{6}$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.250000	2.500000e-01	2.000000e-01
4	1.423611	1.736111e-01	1.219512e-01
8	1.527422	1.038109e-01	6.796480e-02
16	1.584347	5.692448e-02	3.592931e-02
32	1.614167	2.982073e-02	1.847437e-02
64	1.629431	1.526324e-02	9.367223e-03
128	1.637152	7.721504e-03	4.716424e-03
256	1.641035	3.883431e-03	2.366452e-03

iv. Tabla de resultados para aproximación de $\frac{\pi^2}{8}$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.111111	1.111111e-01	1.000000e-01
4	1.171519	6.040816e-02	5.156395e-02
8	1.202491	3.097175e-02	2.575632e-02
16	1.218081	1.558961e-02	1.279850e-02
32	1.225889	7.808056e-03	6.369303e-03
64	1.229794	3.905694e-03	3.175892e-03
128	1.231747	1.953055e-03	1.585597e-03
256	1.232724	9.765538e-04	7.921918e-04

v. Tabla de resultados para aproximación de $\frac{1}{2}$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.400000	6.666667e-02	1.666667e-01
4	0.444444	4.444444e-02	1.000000e-01
8	0.470588	2.614379e-02	5.555556e-02
16	0.484848	1.426025e-02	2.941176e-02
32	0.492308	7.459207e-03	1.515152e-02
64	0.496124	3.816339e-03	7.692308e-03
128	0.498054	1.930444e-03	3.875969e-03
256	0.499025	9.708664e-04	1.945525e-03

vi. Tabla de resultados para aproximación de $\frac{3}{4}$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.458333	1.250000e-01	2.727273e-01
4	0.566667	1.083333e-01	1.911765e-01
8	0.644444	7.777778e-02	1.206897e-01
16	0.692810	4.836601e-02	6.981132e-02
32	0.720143	2.733214e-02	3.795380e-02
64	0.734732	1.458933e-02	1.985667e-02
128	0.742278	7.545942e-03	1.016593e-02
256	0.746116	3.838613e-03	5.144791e-03

vii. Tabla de resultados para aproximación de e^x :

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.500000	5.000000e-01	3.333333e-01
4	1.645833	1.458333e-01	8.860759e-02
8	1.648721	2.887835e-03	1.751560e-03
16	1.648721	1.025454e-07	6.219691e-08
32	1.648721	0.000000e+00	0.000000e+00
64	1.648721	0.000000e+00	0.000000e+00
128	1.648721	0.000000e+00	0.000000e+00
256	1.648721	0.000000e+00	0.000000e+00

- $x = 1.0$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	2.000000	1.000000e+00	5.000000e-01
4	2.666667	6.666667e-01	2.500000e-01
8	2.718254	5.158730e-02	1.897810e-02
16	2.718282	2.786021e-05	1.024920e-05
32	2.718282	5.062617e-14	1.862433e-14
64	2.718282	0.000000e+00	0.000000e+00
128	2.718282	0.000000e+00	0.000000e+00
256	2.718282	0.000000e+00	0.000000e+00

- $x = 1.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	2.500000	1.500000e+00	6.000000e-01
4	4.187500	1.687500e+00	4.029851e-01
8	4.480929	2.934291e-01	6.548399e-02
16	4.481689	7.599408e-04	1.695657e-04
32	4.481689	3.441514e-11	7.679055e-12
64	4.481689	0.000000e+00	0.000000e+00
128	4.481689	0.000000e+00	0.000000e+00
256	4.481689	0.000000e+00	0.000000e+00

viii. Tabla de resultados para aproximación de xe^x :

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.750000	2.500000e-01	3.333333e-01
4	0.822917	7.291667e-02	8.860759e-02
8	0.824361	1.443917e-03	1.751560e-03
16	0.824361	5.127268e-08	6.219691e-08
32	0.824361	0.000000e+00	0.000000e+00
64	0.824361	0.000000e+00	0.000000e+00
128	0.824361	0.000000e+00	0.000000e+00
256	0.824361	0.000000e+00	0.000000e+00

- $x = 1.0$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	2.000000	1.000000e+00	5.000000e-01
4	2.666667	6.666667e-01	2.500000e-01
8	2.718254	5.158730e-02	1.897810e-02
16	2.718282	2.786021e-05	1.024920e-05
32	2.718282	5.062617e-14	1.862433e-14
64	2.718282	0.000000e+00	0.000000e+00
128	2.718282	0.000000e+00	0.000000e+00
256	2.718282	0.000000e+00	0.000000e+00

- $x = 1.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	3.750000	2.250000e+00	6.000000e-01
4	6.281250	2.531250e+00	4.029851e-01
8	6.721394	4.401437e-01	6.548399e-02
16	6.722534	1.139911e-03	1.695657e-04
32	6.722534	5.162093e-11	7.678791e-12
64	6.722534	0.000000e+00	0.000000e+00
128	6.722534	0.000000e+00	0.000000e+00
256	6.722534	0.000000e+00	0.000000e+00

ix. Tabla de resultados para aproximación de $(x + x^2)e^x$:

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.000000	5.000000e-01	5.000000e-01
4	1.229167	2.291667e-01	1.864407e-01
8	1.236540	7.373822e-03	5.963268e-03
16	1.236541	4.644338e-07	3.755911e-07
32	1.236541	0.000000e+00	0.000000e+00
64	1.236541	0.000000e+00	0.000000e+00
128	1.236541	0.000000e+00	0.000000e+00
256	1.236541	0.000000e+00	0.000000e+00

- $x = 1.0$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	3.000000	2.000000e+00	6.666667e-01
4	5.166667	2.166667e+00	4.193548e-01
8	5.436310	2.696429e-01	4.960035e-02
16	5.436564	2.541331e-04	4.674517e-05
32	5.436564	8.659740e-13	1.592870e-13
64	5.436564	0.000000e+00	0.000000e+00
128	5.436564	0.000000e+00	0.000000e+00
256	5.436564	0.000000e+00	0.000000e+00

- $x = 1.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	6.000000	4.500000e+00	7.500000e-01
4	14.437500	8.437500e+00	5.844156e-01
8	16.795857	2.358357e+00	1.404130e-01
16	16.806334	1.047743e-02	6.234214e-04
32	16.806334	8.825012e-10	5.251003e-11
64	16.806334	0.000000e+00	0.000000e+00
128	16.806334	0.000000e+00	0.000000e+00
256	16.806334	0.000000e+00	0.000000e+00

x. Tabla de resultados para aproximación de $\ln(1+x)$:

- $x = -0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	-0.625000	1.250000e-01	2.000000e-01
4	-0.682292	5.729167e-02	8.396947e-02
8	-0.692750	1.045852e-02	1.509710e-02
16	-0.693146	3.961423e-04	5.715132e-04
32	-0.693147	8.522881e-07	1.229592e-06
64	-0.693147	6.858958e-12	9.895384e-12
128	-0.693147	0.000000e+00	0.000000e+00
256	-0.693147	0.000000e+00	0.000000e+00

- $x = 0$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.000000	0.000000e+00	0.000000e+00
4	0.000000	0.000000e+00	0.000000e+00
8	0.000000	0.000000e+00	0.000000e+00
16	0.000000	0.000000e+00	0.000000e+00
32	0.000000	0.000000e+00	0.000000e+00
64	0.000000	0.000000e+00	0.000000e+00
128	0.000000	0.000000e+00	0.000000e+00
256	0.000000	0.000000e+00	0.000000e+00

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.375000	1.250000e-01	3.333333e-01
4	0.401042	2.604167e-02	6.493506e-02
8	0.405315	4.273624e-03	1.054395e-02
16	0.405465	1.495130e-04	3.687447e-04
32	0.405465	3.049350e-07	7.520621e-07
64	0.405465	2.375322e-12	5.858265e-12
128	0.405465	0.000000e+00	0.000000e+00
256	0.405465	0.000000e+00	0.000000e+00

xi. Tabla de resultados para aproximación de $\frac{1}{2} \ln \left(\frac{1+x}{1-x} \right)$:

- x = - 0.5:

n	S _n (x)	Delta _n (x)	e _a (x)
2	-1.083333	8.333333e-02	7.692308e-02
4	-1.098065	1.473214e-02	1.341645e-02
8	-1.098611	5.456552e-04	4.966773e-04
16	-1.098612	1.157223e-06	1.053350e-06
32	-1.098612	9.234169e-12	8.405303e-12
64	-1.098612	0.000000e+00	0.000000e+00
128	-1.098612	0.000000e+00	0.000000e+00
256	-1.098612	0.000000e+00	0.000000e+00

- x = 0:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.000000	0.000000e+00	0.000000e+00
4	0.000000	0.000000e+00	0.000000e+00
8	0.000000	0.000000e+00	0.000000e+00
16	0.000000	0.000000e+00	0.000000e+00
32	0.000000	0.000000e+00	0.000000e+00
64	0.000000	0.000000e+00	0.000000e+00
128	0.000000	0.000000e+00	0.000000e+00
256	0.000000	0.000000e+00	0.000000e+00

- x = 0.5:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.083333	8.333333e-02	7.692308e-02
4	1.098065	1.473214e-02	1.341645e-02
8	1.098611	5.456552e-04	4.966773e-04
16	1.098612	1.157223e-06	1.053350e-06
32	1.098612	9.234169e-12	8.405303e-12
64	1.098612	0.000000e+00	0.000000e+00
128	1.098612	0.000000e+00	0.000000e+00
256	1.098612	0.000000e+00	0.000000e+00

xii. Tabla de resultados para aproximación de $\ln(x)$ versión 1:

- x = 2:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.691358	2.469136e-02	3.571429e-02
4	0.693135	1.776733e-03	2.563329e-03
8	0.693147	1.242222e-05	1.792147e-05
16	0.693147	1.011704e-09	1.459580e-09
32	0.693147	0.000000e+00	0.000000e+00
64	0.693147	0.000000e+00	0.000000e+00
128	0.693147	0.000000e+00	0.000000e+00
256	0.693147	0.000000e+00	0.000000e+00

- x = 4:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.344000	1.440000e-01	1.071429e-01
4	1.383102	3.910217e-02	2.827135e-02
8	1.386265	3.162744e-03	2.281486e-03
16	1.386294	2.944121e-05	2.123734e-05
32	1.386294	4.384766e-09	3.162940e-09
64	1.386294	2.220446e-16	1.601713e-16
128	1.386294	0.000000e+00	0.000000e+00
256	1.386294	0.000000e+00	0.000000e+00

- x = 6:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.671526	2.429543e-01	1.453488e-01
4	1.773004	1.014779e-01	5.723503e-02
8	1.791044	1.804081e-02	1.007279e-02
16	1.791758	7.132236e-04	3.980580e-04
32	1.791759	1.763159e-06	9.840377e-07
64	1.791759	1.935674e-11	1.080320e-11
128	1.791759	0.000000e+00	0.000000e+00
256	1.791759	0.000000e+00	0.000000e+00

xiii. Tabla de resultados para aproximación de $\ln(x)$ versión 2:

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	-0.500000	5.000000e-01	1.000000e+00
4	-0.583333	8.333333e-02	1.428571e-01
8	-0.634524	5.119048e-02	8.067542e-02
16	-0.662872	2.834804e-02	4.276549e-02
32	-0.677766	1.489435e-02	2.197565e-02
64	-0.685396	7.629506e-03	1.113153e-02
128	-0.689256	3.860481e-03	5.600937e-03
256	-0.691198	1.941681e-03	2.809154e-03

- $x = 1.0$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.000000	0.000000e+00	0.000000e+00
4	0.000000	0.000000e+00	0.000000e+00
8	0.000000	0.000000e+00	0.000000e+00
16	0.000000	0.000000e+00	0.000000e+00
32	0.000000	0.000000e+00	0.000000e+00
64	0.000000	0.000000e+00	0.000000e+00
128	0.000000	0.000000e+00	0.000000e+00
256	0.000000	0.000000e+00	0.000000e+00

- $x = 1.5$:

Bloque: x = 1.500000			
=====			
n	S_n(x)	Delta_n(x)	e_a(x)

2	0.388889	5.555556e-02	1.428571e-01
4	0.404321	1.543210e-02	3.816794e-02
8	0.405457	1.136042e-03	2.801880e-03
16	0.405465	8.077809e-06	1.992233e-05
32	0.405465	6.652033e-10	1.640593e-09
64	0.405465	0.000000e+00	0.000000e+00
128	0.405465	0.000000e+00	0.000000e+00
256	0.405465	0.000000e+00	0.000000e+00

xiv. Tabla de resultados para aproximación de $(1+x)^\alpha$:

- x = 0.25, $\alpha = 0.5$:

n	S_n(x)	Delta_n(x)	e_a(x)

2	1.125000	1.250000e-01	1.111111e-01
4	1.118164	6.835938e-03	6.113537e-03
8	1.118034	1.299083e-04	1.161935e-04
16	1.118034	1.654260e-07	1.479615e-07
32	1.118034	8.562040e-13	7.658121e-13
64	1.118034	0.000000e+00	0.000000e+00
128	1.118034	0.000000e+00	0.000000e+00
256	1.118034	0.000000e+00	0.000000e+00

- x = 0.5, $\alpha = 0.5$:

n	S_n(x)	Delta_n(x)	e_a(x)

2	1.250000	2.500000e-01	2.000000e-01
4	1.226562	2.343750e-02	1.910828e-02
8	1.224781	1.781464e-03	1.454516e-03
16	1.224745	3.611764e-05	2.948993e-05
32	1.224745	4.734116e-08	3.865389e-08
64	1.224745	2.484679e-13	2.028732e-13
128	1.224745	0.000000e+00	0.000000e+00
256	1.224745	0.000000e+00	0.000000e+00

- x = 0.75, $\alpha = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.375000	3.750000e-01	2.727273e-01
4	1.331055	4.394531e-02	3.301541e-02
8	1.323685	7.369906e-03	5.567720e-03
16	1.322903	7.822297e-04	5.912981e-04
32	1.322876	2.680420e-05	2.026207e-05
64	1.322876	9.233631e-08	6.979969e-08
128	1.322876	3.228084e-12	2.440202e-12
256	1.322876	0.000000e+00	0.000000e+00

xv. Tabla de resultados para aproximación de a^x :

- x = 0.5, a = 2:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.346574	3.465736e-01	2.573744e-01
4	1.413568	6.699464e-02	4.739399e-02
8	1.414214	6.453264e-04	4.563147e-04
16	1.414214	5.368203e-09	3.795893e-09
32	1.414214	0.000000e+00	0.000000e+00
64	1.414214	0.000000e+00	0.000000e+00
128	1.414214	0.000000e+00	0.000000e+00
256	1.414214	0.000000e+00	0.000000e+00

- x = 1.0, a = 2:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.693147	6.931472e-01	4.093839e-01
4	1.988878	2.957306e-01	1.486922e-01
8	1.999999	1.112077e-02	5.560390e-03
16	2.000000	1.430856e-06	7.154281e-07
32	2.000000	2.220446e-16	1.110223e-16
64	2.000000	0.000000e+00	0.000000e+00
128	2.000000	0.000000e+00	0.000000e+00
256	2.000000	0.000000e+00	0.000000e+00

- x = 1.5, a = 2:

n	S _n (x)	Delta _n (x)	e _a (x)
2	2.039721	1.039721e+00	5.097368e-01
4	2.767557	7.278360e-01	2.629886e-01
8	2.828389	6.083211e-02	2.150769e-02
16	2.828427	3.823154e-05	1.351689e-05
32	2.828427	9.503509e-14	3.359998e-14
64	2.828427	0.000000e+00	0.000000e+00
128	2.828427	0.000000e+00	0.000000e+00
256	2.828427	0.000000e+00	0.000000e+00

xvi. Tabla de resultados para aproximación de B_k :

La ejecución de la Serie 16 en el nivel 2 se truncó deliberadamente en la octava n=8 por la activación de los protocolos de seguridad de memoria contra desbordamiento recursivo.

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.166667	6.666667e-01	4.000000e+00
4	-0.033333	2.000000e-01	6.000000e+00
8	-0.033333	1.942890e-16	5.828671e-15
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
16	0.000000	3.333333e-02	3.333333e+13
Alerta [Serie 16]: k=32 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
32	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 16]: k=64 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=32 excedera la memoria por recursion. Abortando.			
64	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 16]: k=128 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=64 excedera la memoria por recursion. Abortando.			
128	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 16]: k=256 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=128 excedera la memoria por recursion. Abortando.			
256	0.000000	0.000000e+00	0.000000e+00

xvii. Tabla de resultados para aproximación de E_k :

La ejecución de la Serie 17 en el nivel 2 se truncó deliberadamente en la octava n=16 por la activación de los protocolos de seguridad de memoria contra desbordamiento recursivo.

n	S _n (x)	Delta _n (x)	e _a (x)

2	5.000000	4.000001e+00	8.000001e-01
4	1385.000000	1.380000e+03	9.963899e-01
8	19391512145.000019	1.939151e+10	9.999999e-01
16	177519391579539515613600808960.000000	1.775194e+29	1.000000e+00
Alerta [Serie 17]: Limite computacional excedido.			
32	0.000000	1.775194e+29	1.775194e+44
Alerta [Serie 17]: Limite computacional excedido.			
Alerta [Serie 17]: Limite computacional excedido.			
64	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 17]: Limite computacional excedido.			
Alerta [Serie 17]: Limite computacional excedido.			
128	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 17]: Limite computacional excedido.			
Alerta [Serie 17]: Limite computacional excedido.			
256	0.000000	0.000000e+00	0.000000e+00

xviii. Tabla de resultados para aproximación de E_{2k} :

La ejecución de la Serie 18 en el nivel 2 se truncó deliberadamente en la octava n=8 por la activación de los protocolos de seguridad de memoria contra desbordamiento recursivo.

n	S _n (x)	Delta _n (x)	e _a (x)

2	5.000000	6.000000e+00	1.200000e+00
4	1385.000000	1.380000e+03	9.963899e-01
8	19391512144.945312	1.939151e+10	9.999999e-01
Alerta [Serie 18]: Limite computacional excedido.			
16	0.000000	1.939151e+10	1.939151e+25
Alerta [Serie 18]: Limite computacional excedido.			
Alerta [Serie 18]: Limite computacional excedido.			
32	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 18]: Limite computacional excedido.			
Alerta [Serie 18]: Limite computacional excedido.			
64	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 18]: Limite computacional excedido.			
Alerta [Serie 18]: Limite computacional excedido.			
128	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 18]: Limite computacional excedido.			
Alerta [Serie 18]: Limite computacional excedido.			
256	0.000000	0.000000e+00	0.000000e+00

xix. Tabla de resultados para aproximación de $\text{sen}(x)$:

- x = 0.5:

n	S_n(x)	Delta_n(x)	e_a(x)
2	0.479167	2.083333e-02	4.347826e-02
4	0.479426	2.588666e-04	5.399516e-04
8	0.479426	5.370076e-09	1.120106e-08
16	0.479426	0.000000e+00	0.000000e+00
32	0.479426	0.000000e+00	0.000000e+00
64	0.479426	0.000000e+00	0.000000e+00
128	0.479426	0.000000e+00	0.000000e+00
256	0.479426	0.000000e+00	0.000000e+00

- x = 1.0:

n	S_n(x)	Delta_n(x)	e_a(x)
2	0.833333	1.666667e-01	2.000000e-01
4	0.841468	8.134921e-03	9.667531e-03
8	0.841471	2.730840e-06	3.245316e-06
16	0.841471	2.775558e-15	3.298459e-15
32	0.841471	0.000000e+00	0.000000e+00
64	0.841471	0.000000e+00	0.000000e+00
128	0.841471	0.000000e+00	0.000000e+00
256	0.841471	0.000000e+00	0.000000e+00

- x = 1.5:

Bloque: x = 1.500000			
n	S_n(x)	Delta_n(x)	e_a(x)
2	0.937500	5.625000e-01	6.000000e-01
4	0.997391	5.989118e-02	6.004784e-02
8	0.997495	1.038036e-04	1.040642e-04
16	0.997495	2.751910e-12	2.758821e-12
32	0.997495	0.000000e+00	0.000000e+00
64	0.997495	0.000000e+00	0.000000e+00
128	0.997495	0.000000e+00	0.000000e+00
256	0.997495	0.000000e+00	0.000000e+00

xx. Tabla de resultados para aproximación de $\cos(x)$:

- x = 0.5:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.875000	1.250000e-01	1.428571e-01
4	0.877582	2.582465e-03	2.942704e-03
8	0.877583	9.661260e-08	1.100895e-07
16	0.877583	0.000000e+00	0.000000e+00
32	0.877583	0.000000e+00	0.000000e+00
64	0.877583	0.000000e+00	0.000000e+00
128	0.877583	0.000000e+00	0.000000e+00
256	0.877583	0.000000e+00	0.000000e+00

- x = 1.0:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.500000	5.000000e-01	1.000000e+00
4	0.540278	4.027778e-02	7.455013e-02
8	0.540302	2.452809e-05	4.539698e-05
16	0.540302	4.762857e-14	8.815170e-14
32	0.540302	0.000000e+00	0.000000e+00
64	0.540302	0.000000e+00	0.000000e+00
128	0.540302	0.000000e+00	0.000000e+00
256	0.540302	0.000000e+00	0.000000e+00

- x = 1.5:

n	S _n (x)	Delta _n (x)	e _a (x)
2	-0.125000	1.125000e+00	9.000000e+00
4	0.070117	1.951172e-01	2.782730e+00
8	0.070737	6.200141e-04	8.765036e-03
16	0.070737	3.116409e-11	4.405615e-10
32	0.070737	0.000000e+00	0.000000e+00
64	0.070737	0.000000e+00	0.000000e+00
128	0.070737	0.000000e+00	0.000000e+00
256	0.070737	0.000000e+00	0.000000e+00

xxi. Tabla de resultados para aproximación de $\tan(x)$:

La función tangente no es independiente, sino que requiere la evaluación de los números de Bernoulli (serie 16) para sus coeficientes. Al intentar calcular el n=16, el volumen de llamadas apiladas en memoria amenaza con causar un desbordamiento.

- $x = 0.25$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.255208	5.208333e-03	2.040816e-02
4	0.255342	1.335023e-04	5.228375e-04
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
8	0.255342	8.559217e-08	3.352061e-07
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=18 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=20 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=22 excedera la memoria por recursion. Abortando.			

xxii. Tabla de resultados para aproximación de $\sec(x)$:

La función secante no es independiente, sino que requiere la evaluación de los números de Euler (serie 18) para sus coeficientes. Al intentar calcular el $n=32$, el volumen de llamadas apiladas en memoria amenaza con causar un desbordamiento.

- $x = 0.25$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.031250	3.125000e-02	3.030303e-02
4	1.032084	8.344862e-04	8.085445e-04
8	1.032085	5.377645e-07	5.210467e-07
16	1.032085	2.216005e-13	2.147115e-13
Alerta [Serie 18]: Limite computacional excedido.			
Alerta [Serie 18]: Limite computacional excedido.			
Alerta [Serie 18]: Limite computacional excedido.			

xxiii. Tabla de resultados para aproximación de $\csc(x)$:

La función secante no es independiente, sino que requiere la evaluación de los números de Bernoulli (serie 16) para sus coeficientes. Al intentar calcular el $n=16$, el volumen de llamadas apiladas en memoria amenaza con causar un desbordamiento.

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	2.083333	8.333333e-02	4.000000e-02
4	2.080967	2.366485e-03	1.137204e-03
8	2.080965	1.599879e-06	7.688159e-07
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=18 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=20 excedera la memoria por recursion. Abortando.			

xxiv. Tabla de resultados para aproximación de $\arcsen(x)$:

- x = 0.3:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.304500	4.500000e-03	1.477833e-02
4	0.304692	1.920134e-04	6.301885e-04
8	0.304693	6.406064e-07	2.102467e-06
16	0.304693	1.614781e-11	5.299703e-11
32	0.304693	0.000000e+00	0.000000e+00
64	0.304693	0.000000e+00	0.000000e+00
128	0.304693	0.000000e+00	0.000000e+00
256	0.304693	0.000000e+00	0.000000e+00

- x = 0.6:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.636000	3.600000e-02	5.660377e-02
4	0.643082	7.081714e-03	1.101215e-02
8	0.643498	4.165755e-04	6.473607e-04
16	0.643501	2.818696e-06	4.380251e-06
32	0.643501	3.023536e-10	4.698571e-10
64	0.643501	0.000000e+00	0.000000e+00
128	0.643501	0.000000e+00	0.000000e+00
256	0.643501	0.000000e+00	0.000000e+00

- x = 0.9:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.021500	1.215000e-01	1.189427e-01
4	1.087139	6.563929e-02	6.037799e-02
8	1.113217	2.607729e-02	2.342517e-02
16	1.119245	6.028477e-03	5.386200e-03
32	1.119762	5.171884e-04	4.618734e-04
64	1.119770	7.270010e-06	6.492416e-06
128	1.119770	3.289092e-09	2.937294e-09
256	1.119770	1.332268e-15	1.189770e-15

xxv. Tabla de resultados para aproximación de $\arccos(x)$:

- $x = 0.3$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.266296	4.500000e-03	3.553671e-03
4	1.266104	1.920134e-04	1.516569e-04
8	1.266104	6.406064e-07	5.059668e-07
16	1.266104	1.614797e-11	1.275407e-11
32	1.266104	0.000000e+00	0.000000e+00
64	1.266104	0.000000e+00	0.000000e+00
128	1.266104	0.000000e+00	0.000000e+00
256	1.266104	0.000000e+00	0.000000e+00

- $x = 0.6$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.934796	3.600000e-02	3.851106e-02
4	0.927715	7.081714e-03	7.633505e-03
8	0.927298	4.165755e-04	4.492358e-04
16	0.927295	2.818696e-06	3.039697e-06
32	0.927295	3.023536e-10	3.260597e-10
64	0.927295	0.000000e+00	0.000000e+00
128	0.927295	0.000000e+00	0.000000e+00
256	0.927295	0.000000e+00	0.000000e+00

- $x = 0.9$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.549296	1.215000e-01	2.211921e-01
4	0.483657	6.563929e-02	1.357145e-01
8	0.457580	2.607729e-02	5.698960e-02
16	0.451551	6.028477e-03	1.335059e-02
32	0.451034	5.171884e-04	1.146673e-03
64	0.451027	7.270010e-06	1.611880e-05
128	0.451027	3.289092e-09	7.292454e-09
256	0.451027	1.332268e-15	2.953855e-15

xxvi. Tabla de resultados para aproximación de $\arctan(x)$:

- $x = 0.25$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.244792	5.208333e-03	2.127660e-02
4	0.244978	1.865932e-04	7.616725e-04
8	0.244979	4.032650e-07	1.646123e-06
16	0.244979	3.242739e-12	1.323682e-11
32	0.244979	0.000000e+00	0.000000e+00
64	0.244979	0.000000e+00	0.000000e+00
128	0.244979	0.000000e+00	0.000000e+00
256	0.244979	0.000000e+00	0.000000e+00

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.458333	4.166667e-02	9.090909e-02
4	0.463467	5.133929e-03	1.107722e-02
8	0.463647	1.799802e-04	3.881835e-04
16	0.463648	3.668900e-07	7.913122e-07
32	0.463648	2.855105e-12	6.157920e-12
64	0.463648	0.000000e+00	0.000000e+00
128	0.463648	0.000000e+00	0.000000e+00
256	0.463648	0.000000e+00	0.000000e+00

- $x = 0.8$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.629333	1.706667e-01	2.711864e-01
4	0.664910	3.557669e-02	5.350602e-02
8	0.673897	8.987316e-03	1.333633e-02
16	0.674729	8.316233e-04	1.232529e-03
32	0.674741	1.197961e-05	1.775438e-05
64	0.674741	4.766929e-09	7.064829e-09
128	0.674741	1.443290e-15	2.139028e-15
256	0.674741	0.000000e+00	0.000000e+00

xxvii. Tabla de resultados para aproximación de $\sinh(x)$:

- $x = 0.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.520833	2.083333e-02	4.000000e-02
4	0.521095	2.619668e-04	5.027233e-04
8	0.521095	5.394541e-09	1.035231e-08
16	0.521095	0.000000e+00	0.000000e+00
32	0.521095	0.000000e+00	0.000000e+00
64	0.521095	0.000000e+00	0.000000e+00
128	0.521095	0.000000e+00	0.000000e+00
256	0.521095	0.000000e+00	0.000000e+00

- $x = 1.5$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	2.062500	5.625000e-01	2.727273e-01
4	2.129171	6.667132e-02	3.131327e-02
8	2.129279	1.081381e-04	5.078625e-05
16	2.129279	2.788436e-12	1.309568e-12
32	2.129279	0.000000e+00	0.000000e+00
64	2.129279	0.000000e+00	0.000000e+00
128	2.129279	0.000000e+00	0.000000e+00
256	2.129279	0.000000e+00	0.000000e+00

- $x = 2.5$:

Bloque: x = 2.500000			
=====			
n	S_n(x)	Delta_n(x)	e_a(x)

2	5.104167	2.604167e+00	5.102041e-01
4	6.039070	9.349036e-01	1.548092e-01
8	6.050204	1.113421e-02	1.840304e-03
16	6.050204	1.666841e-08	2.755015e-09
32	6.050204	0.000000e+00	0.000000e+00
64	6.050204	0.000000e+00	0.000000e+00
128	6.050204	0.000000e+00	0.000000e+00
256	6.050204	0.000000e+00	0.000000e+00

xxviii. Tabla de resultados para aproximación de $\cosh(x)$:

- x = 0.5:

Bloque: x = 0.500000			
=====			
n	S_n(x)	Delta_n(x)	e_a(x)

2	1.125000	1.250000e-01	1.111111e-01
4	1.127626	2.625868e-03	2.328670e-03
8	1.127626	9.715083e-08	8.615519e-08
16	1.127626	0.000000e+00	0.000000e+00
32	1.127626	0.000000e+00	0.000000e+00
64	1.127626	0.000000e+00	0.000000e+00
128	1.127626	0.000000e+00	0.000000e+00
256	1.127626	0.000000e+00	0.000000e+00

- x = 1.5:

Bloque: x = 1.500000			
=====			
n	S_n(x)	Delta_n(x)	e_a(x)

2	2.125000	1.125000e+00	5.294118e-01
4	2.351758	2.267578e-01	9.642056e-02
8	2.352410	6.518027e-04	2.770787e-04
16	2.352410	3.162581e-11	1.344401e-11
32	2.352410	0.000000e+00	0.000000e+00
64	2.352410	0.000000e+00	0.000000e+00
128	2.352410	0.000000e+00	0.000000e+00
256	2.352410	0.000000e+00	0.000000e+00

- x = 2.5:

Bloque: x = 2.500000			
=====			
n	S_n(x)	Delta_n(x)	e_a(x)

2	4.125000	3.125000e+00	7.575758e-01
4	6.091688	1.966688e+00	3.228478e-01
8	6.132289	4.060100e-02	6.620855e-03
16	6.132289	1.135917e-07	1.852353e-08
32	6.132289	0.000000e+00	0.000000e+00
64	6.132289	0.000000e+00	0.000000e+00
128	6.132289	0.000000e+00	0.000000e+00
256	6.132289	0.000000e+00	0.000000e+00

xxix. Tabla de resultados para aproximación de $\tanh(x)$:

La función tangente hiperbólica no es independiente, sino que requiere la evaluación de los números de Euler (serie 18) para sus coeficientes. Al intentar calcular el $n=32$, el volumen de llamadas apiladas en memoria amenaza con causar un desbordamiento.

- x = 0.3:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.291000	9.000000e-03	3.092784e-02
4	0.291312	3.121971e-04	1.071693e-03
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
8	0.291313	4.153289e-07	1.425715e-06
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=18 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=20 excedera la memoria por recursion. Abortando.			

xxx. Tabla de resultados para aproximación de $\operatorname{arcsenh}(x)$:

- x = 0.25:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.247396	2.604167e-03	1.052632e-02
4	0.247466	7.051740e-05	2.849575e-04
8	0.247466	1.108096e-07	4.477760e-07
16	0.247466	6.387113e-13	2.581001e-12
32	0.247466	0.000000e+00	0.000000e+00
64	0.247466	0.000000e+00	0.000000e+00
128	0.247466	0.000000e+00	0.000000e+00
256	0.247466	0.000000e+00	0.000000e+00

- x = 0.5:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.479167	2.083333e-02	4.347826e-02
4	0.481162	1.994978e-03	4.146169e-03
8	0.481212	5.010791e-05	1.041286e-04
16	0.481212	7.280036e-08	1.512855e-07
32	0.481212	4.018452e-13	8.350693e-13
64	0.481212	0.000000e+00	0.000000e+00
128	0.481212	0.000000e+00	0.000000e+00
256	0.481212	0.000000e+00	0.000000e+00

- x = 0.8:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.714667	8.533333e-02	1.194030e-01
4	0.729880	1.521371e-02	2.084412e-02
8	0.732499	2.618676e-03	3.574989e-03
16	0.732667	1.675030e-04	2.286210e-04
32	0.732668	1.695883e-06	2.314667e-06
64	0.732668	4.763744e-10	6.501911e-10
128	0.732668	2.220446e-16	3.030630e-16
256	0.732668	0.000000e+00	0.000000e+00

xxxi. Tabla de resultados para aproximación de $\arctanh(x)$:

- $x = 0.3$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.309000	9.000000e-03	2.912621e-02
4	0.309517	5.172429e-04	1.671128e-03
8	0.309520	2.361263e-06	7.628801e-06
16	0.309520	8.262463e-11	2.669447e-10
32	0.309520	0.000000e+00	0.000000e+00
64	0.309520	0.000000e+00	0.000000e+00
128	0.309520	0.000000e+00	0.000000e+00
256	0.309520	0.000000e+00	0.000000e+00

- $x = 0.6$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.672000	7.200000e-02	1.071429e-01
4	0.691551	1.955109e-02	2.827135e-02
8	0.693132	1.581372e-03	2.281486e-03
16	0.693147	1.472061e-05	2.123734e-05
32	0.693147	2.192383e-09	3.162940e-09
64	0.693147	1.110223e-16	1.601713e-16
128	0.693147	0.000000e+00	0.000000e+00
256	0.693147	0.000000e+00	0.000000e+00

- $x = 0.9$:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.143000	2.430000e-01	2.125984e-01
4	1.329426	1.864261e-01	1.402305e-01
8	1.434280	1.048538e-01	7.310556e-02
16	1.468144	3.386391e-02	2.306580e-02
32	1.472142	3.998592e-03	2.716172e-03
64	1.472219	7.696310e-05	5.227692e-05
128	1.472219	4.808200e-08	3.265953e-08
256	1.472219	3.375078e-14	2.292510e-14

xxxii. Tabla de resultados para aproximación de $\frac{\ln(1+x)}{1+x}$:

- x = 0.2:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.140000	6.000000e-02	4.285714e-01
4	0.151333	1.133333e-02	7.488987e-02
8	0.151933	6.000975e-04	3.949740e-03
16	0.151935	1.199801e-06	7.896822e-06
32	0.151935	3.746670e-12	2.465975e-11
64	0.151935	0.000000e+00	0.000000e+00
128	0.151935	0.000000e+00	0.000000e+00
256	0.151935	0.000000e+00	0.000000e+00

- x = 0.5:

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.125000	3.750000e-01	3.000000e+00
4	0.223958	9.895833e-02	4.418605e-01
8	0.266671	4.271298e-02	1.601709e-01
16	0.270293	3.621357e-03	1.339791e-02
32	0.270310	1.739825e-05	6.436405e-05
64	0.270310	3.165642e-10	1.171115e-09
128	0.270310	0.000000e+00	0.000000e+00
256	0.270310	0.000000e+00	0.000000e+00

- x = 0.8:

n	S _n (x)	Delta _n (x)	e _a (x)
2	-0.160000	9.600000e-01	6.000000e+00
4	-0.074667	8.533333e-02	1.142857e+00
8	0.119064	1.937307e-01	1.627114e+00
16	0.283836	1.647718e-01	5.805181e-01
32	0.325113	4.127723e-02	1.269627e-01
64	0.326547	1.433780e-03	4.390733e-03
128	0.326548	1.325861e-06	4.060232e-06
256	0.326548	9.524048e-13	2.916583e-12

xxxiii. Tabla de resultados para aproximación de $e^{\sin(x)}$:

- x = 0.5:

n	S _n (x)	Delta _n (x)	e _a (x)
2	1.500000	5.000000e-01	3.333333e-01
4	1.625000	1.250000e-01	7.692308e-02
8	1.615104	9.895833e-03	6.127056e-03
16	1.615104	0.000000e+00	0.000000e+00
32	1.615104	0.000000e+00	0.000000e+00
64	1.615104	0.000000e+00	0.000000e+00
128	1.615104	0.000000e+00	0.000000e+00
256	1.615104	0.000000e+00	0.000000e+00

- x = 1.0:

n	S _n (x)	Delta _n (x)	e _a (x)
2	2.000000	1.000000e+00	5.000000e-01
4	2.500000	5.000000e-01	2.000000e-01
8	2.308333	1.916667e-01	8.303249e-02
16	2.308333	0.000000e+00	0.000000e+00
32	2.308333	0.000000e+00	0.000000e+00
64	2.308333	0.000000e+00	0.000000e+00
128	2.308333	0.000000e+00	0.000000e+00
256	2.308333	0.000000e+00	0.000000e+00

- x = 2.0:

n	S _n (x)	Delta _n (x)	e _a (x)
2	3.000000	2.000000e+00	6.666667e-01
4	5.000000	2.000000e+00	4.000000e-01
8	0.866667	4.133333e+00	4.769231e+00
16	0.866667	0.000000e+00	0.000000e+00
32	0.866667	0.000000e+00	0.000000e+00
64	0.866667	0.000000e+00	0.000000e+00
128	0.866667	0.000000e+00	0.000000e+00
256	0.866667	0.000000e+00	0.000000e+00

3. Análisis de Exactitud y Limitaciones Numéricas

Aunque las series de Taylor constituyen representaciones matemáticamente exactas de funciones, su implementación en computadora está sujeta a las limitaciones de la aritmética en punto flotante. Debido a que los números se almacenan con una precisión finita, surgen errores de redondeo, overflow y pérdida de significancia que afectan directamente la exactitud de los resultados obtenidos.

Un aspecto fundamental en este contexto es la elección del tipo de dato. El tipo float, con una representación de 32 bits, ofrece aproximadamente 7 dígitos de precisión decimal, mientras que el tipo double, con 64 bits, alcanza entre 15 y 16 dígitos. Esta diferencia resulta crítica al evaluar series que involucran factoriales en el denominador, como en el caso de la función exponencial o trigonométrica. Dado que el crecimiento de la factorial es extremadamente rápido, el uso de float provoca overflow en pocas iteraciones, generando valores infinitos (∞) o indefinidos (NaN). Por su parte, el tipo double permite extender el rango de cálculo antes de que ocurra este fenómeno, aunque no lo elimina por completo, evidenciando una limitación inherente de la representación numérica.

```
Resultados para e^10.0 con n=40 iteraciones:
```

```
-----  
Resultado FLOAT (32 bits):  nan  
Resultado DOUBLE (64 bits): 22026.465795
```

```
Resultados para e^10.0 con n=34 iteraciones:
```

```
-----  
Resultado FLOAT (32 bits): 22026.466797  
Resultado DOUBLE (64 bits): 22026.465748
```

Durante las pruebas experimentales con e^{10} se contrastó la precisión de ambos tipos de datos. Mientras el double mantuvo la integridad de los decimales, el float mostró una desviación visible a partir del séptimo dígito significativo (variando de .748 en double a .797 en float). Esto demuestra que para el cálculo de series de Taylor, el float no solo es propenso al overflow en los denominadores, sino que sufre de una degradación de precisión temprana debido a su mantisa limitada de 23 bits, lo que lo inhabilita para cálculos de alta precisión en ingeniería.

Por otra parte, es indispensable considerar las restricciones del dominio de cada función para evitar resultados inválidos. En este sentido, se implementaron guardias de dominio que aseguran que los valores de entrada se mantengan dentro de los intervalos de convergencia o definición matemática. Por ejemplo, en el caso de la serie de $\ln(1+x)$, se verificó que $|x| < 1$ antes de realizar el cálculo, evitando así divergencia o resultados incorrectos. Este tipo de validaciones previene errores como divisiones entre cero o evaluaciones fuera del dominio real de la función.

Ahora bien, además de los problemas de precisión numérica, surgen limitaciones relacionadas con el uso de memoria. Durante las pruebas de rendimiento del Nivel 2, se observó una limitación importante del sistema al calcular los Números de Bernoulli (Serie 16) usando recursión directa. Como se muestra en los resultados, el programa comienza a fallar a partir de $n=16$.

n	S _n (x)	Delta _n (x)	e _a (x)
2	0.166667	6.666667e-01	4.000000e+00
4	-0.033333	2.000000e-01	6.000000e+00
8	-0.033333	1.942890e-16	5.828671e-15
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
16	0.000000	3.333333e-02	3.333333e+13
Alerta [Serie 16]: k=32 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=16 excedera la memoria por recursion. Abortando.			
32	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 16]: k=64 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=32 excedera la memoria por recursion. Abortando.			
64	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 16]: k=128 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=64 excedera la memoria por recursion. Abortando.			
128	0.000000	0.000000e+00	0.000000e+00
Alerta [Serie 16]: k=256 excedera la memoria por recursion. Abortando.			
Alerta [Serie 16]: k=128 excedera la memoria por recursion. Abortando.			
256	0.000000	0.000000e+00	0.000000e+00

Uno de los principales problemas es el uso excesivo de memoria. La forma en que está implementado el cálculo hace que, en cada paso, la función se llame a sí misma varias veces para resolver el problema. Esto provoca que el número de llamadas crezca muy rápido conforme aumenta n , llenando la memoria disponible. Para evitar que el programa se cierre de manera inesperada, se implementó un mecanismo de seguridad que detiene el cálculo y devuelve el valor 0.000000, como se observa en las alertas.

Otro aspecto importante es el alto costo de los cálculos internos. En cada paso se realizan operaciones con coeficientes binomiales $\binom{k}{i}$, los cuales dependen de factoriales. Como las factoriales crecen muy rápido, esto incrementa considerablemente el tiempo de ejecución y el consumo de recursos, haciendo que el método sea poco eficiente para valores grandes de n .

Cabe destacar que este problema no está relacionado con la precisión de los resultados. Por ejemplo, para valores pequeños como $n=8$, el error es extremadamente bajo (del orden de 10^{-15} , lo que indica que el cálculo es muy preciso. Por lo tanto, la falla se debe principalmente a limitaciones en el uso de memoria, y no a errores numéricos.

Este comportamiento justifica la transición al Nivel 3 del proyecto, donde se utiliza la técnica de memorización. Esta consiste en guardar los resultados ya calculados para no repetir operaciones innecesarias. De esta manera, se reduce significativamente el número de cálculos, mejorando el rendimiento y permitiendo trabajar con valores mucho más grandes, como $n = 256$, sin comprometer la estabilidad del sistema.

4. Discusión de Convergencia y Estabilidad

Para poder realizar el análisis necesitamos comprender que el comportamiento de las series está determinado principalmente por tres factores matemáticos que influyen directamente en el desempeño del software en C:

- Naturaleza del denominador: La rapidez con la que una serie converge depende de qué tan velozmente el término general (an) tiende a cero. Aquellas con denominadores factoriales ($n!$) muestran una convergencia muy acelerada porque como sabemos mientras más grande sea el denominador más se acerca a cero, alcanzando la precisión de tipo double en pocas iteraciones. En cambio, las que poseen denominadores polinómicos (nk) presentan una convergencia considerablemente más lenta.
- Radio de convergencia: No todas las series convergen para cualquier valor de entrada. Por ejemplo, funciones como $\text{sen}(x)$ convergen en todo el conjunto de los números reales, mientras que otras, como $\ln(1+x)$ solo lo hacen dentro de un intervalo limitado ($|x| < 1$). Evaluar valores cercanos al límite de este radio incrementa de forma significativa el tiempo de cálculo.
- Estabilidad numérica y coeficientes complejos: Existen series que dependen de coeficientes especiales, como los números de Bernoulli o de Euler. En estos casos, el costo computacional no solo proviene de la suma de términos, sino también del cálculo de dichos coeficientes, que generalmente se obtiene de manera recursiva y añade una carga adicional al procesador.

¿Por qué se comportan diferente?

Se elegirán $x=0.5$, $x=0.95$ y $x=10$ debido a que; 0.5 Está dentro del radio de convergencia de la mayoría de las series. 0.95 está muy cerca del borde del radio de convergencia $|x| < 1$. Y 10 porque es un valor "grande" para una serie de Taylor.

e^x :

Bloque: x = 0.500000				
n	S_n(x)	Delta_n(x)	e_a(x)	Tiempo (us)
2	1.500000	5.000000e-01	3.333333e-01	0.0470
4	1.645833	1.458333e-01	8.860759e-02	0.0440
8	1.648721	2.887835e-03	1.751560e-03	0.0580
16	1.648721	1.025454e-07	6.219691e-08	0.0830
32	1.648721	0.000000e+00	0.000000e+00	0.1520
64	1.648721	0.000000e+00	0.000000e+00	0.2690
128	1.648721	0.000000e+00	0.000000e+00	0.5330
256	1.648721	0.000000e+00	0.000000e+00	1.6070

Bloque: x = 0.950000				
n	S_n(x)	Delta_n(x)	e_a(x)	Tiempo (us)
2	1.950000	9.500000e-01	4.871795e-01	0.0100
4	2.544146	5.941458e-01	2.335345e-01	0.0150
8	2.585691	4.154545e-02	1.606745e-02	0.0330
16	2.585710	1.837114e-05	7.104874e-06	0.0500
32	2.585710	2.220446e-14	8.587376e-15	0.1080
64	2.585710	0.000000e+00	0.000000e+00	0.2520
128	2.585710	0.000000e+00	0.000000e+00	0.4370
256	2.585710	0.000000e+00	0.000000e+00	1.2560

Bloque: x = 10.000000				
n	S_n(x)	Delta_n(x)	e_a(x)	Tiempo (us)
2	11.000000	1.000000e+01	9.090909e-01	0.0120
4	227.666667	2.166667e+02	9.516837e-01	0.0160
8	4850.682540	4.623016e+03	9.530650e-01	0.0320
16	20952.886969	1.610220e+04	7.684957e-01	0.0560
32	22026.465252	1.073578e+03	4.874038e-02	0.1140
64	22026.465795	5.424228e-04	2.462595e-08	0.2180
128	22026.465795	0.000000e+00	0.000000e+00	0.4370
256	22026.465795	0.000000e+00	0.000000e+00	0.8350

$\ln(x + 1)$:

Bloque: x = 0.950000				
n	S_n(x)	Delta_n(x)	e_a(x)	Tiempo (us)
2	0.498750	4.512500e-01	9.047619e-01	0.0120
4	0.580915	8.216510e-02	1.414408e-01	0.0260
8	0.629991	4.907579e-02	7.789920e-02	0.0530
16	0.654856	2.486520e-02	3.797048e-02	0.1350
32	0.664927	1.007135e-02	1.514653e-02	0.2980
64	0.667546	2.618579e-03	3.922695e-03	0.6320
128	0.667824	2.780152e-04	4.163000e-04	1.1170
256	0.667829	5.333975e-06	7.987033e-06	2.3350

Bloque: x = 0.500000				
n	S_n(x)	Delta_n(x)	e_a(x)	Tiempo (us)
2	0.375000	1.250000e-01	3.333333e-01	0.0330
4	0.401042	2.604167e-02	6.493506e-02	0.0550
8	0.405315	4.273624e-03	1.054395e-02	0.0880
16	0.405465	1.495130e-04	3.687447e-04	0.1590
32	0.405465	3.049350e-07	7.520621e-07	0.3060
64	0.405465	2.375322e-12	5.858265e-12	0.6760
128	0.405465	0.000000e+00	0.000000e+00	1.3330
256	0.405465	0.000000e+00	0.000000e+00	2.7130

Bloque: x = 10.000000				
n	S_n(x)	Delta_n(x)	e_a(x)	Tiempo (us)
2	0.000000	0.000000e+00	0.000000e+00	0.0170
4	0.000000	0.000000e+00	0.000000e+00	0.0170
8	0.000000	0.000000e+00	0.000000e+00	0.0150
16	0.000000	0.000000e+00	0.000000e+00	0.0160
32	0.000000	0.000000e+00	0.000000e+00	0.0070
64	0.000000	0.000000e+00	0.000000e+00	0.0060
128	0.000000	0.000000e+00	0.000000e+00	0.0060
256	0.000000	0.000000e+00	0.000000e+00	0.0060

En el presente análisis numérico se evaluó el comportamiento de las series de Taylor asociadas a las funciones e^x y $\ln(1+x)$, observándose una clara correspondencia entre los resultados experimentales y la teoría matemática.

En el caso de la función e^x , los resultados muestran una convergencia rápida y estable para valores pequeños y moderados de x, lo cual se explica por la presencia de términos factoriales en el denominador de la serie, que provocan una disminución acelerada de los términos. Sin embargo, para valores grandes de x, aunque la serie continúa convergiendo debido a su radio de convergencia infinito, se detecta un incremento significativo en el número de iteraciones necesarias y posibles efectos de inestabilidad numérica, como crecimiento excesivo de términos intermedios y pérdida de precisión.

Por otro lado, la serie de $\ln(1+x)$ presenta un comportamiento más restrictivo, ya que su convergencia está limitada al intervalo $|x| < 1$. Esto se confirma en los resultados: para valores dentro de dicho intervalo, la serie converge correctamente, aunque de forma más lenta en comparación con e^x , especialmente cuando x se aproxima al límite del radio de convergencia. En

contraste, para valores fuera de este intervalo, como $x = 10$, la serie deja de ser válida, produciendo resultados incorrectos o nulos, lo cual evidencia la importancia de respetar las condiciones teóricas de convergencia.

En conjunto, los resultados obtenidos no solo validan el comportamiento teórico de ambas series, sino que también demuestran la influencia de factores como la naturaleza del denominador, el radio de convergencia y la estabilidad numérica en la eficiencia y precisión de los métodos implementados en lenguaje C.

Para evidenciar los errores de cancelación y la pérdida de datos significativos, se empleará la función $\sin(x)$. Esta función resulta adecuada para el análisis debido a que su desarrollo en serie de Taylor presenta términos alternantes, lo que favorece la aparición de cancelaciones numéricas cuando se trabaja con valores grandes de x o con una cantidad elevada de iteraciones.

Bloque: $x = 0.500000$			
n	$S_n(x)$	$\Delta_n(x)$	$e_a(x)$
2	0.479167	2.083333e-02	4.347826e-02
4	0.479426	2.588666e-04	5.399516e-04
8	0.479426	5.370076e-09	1.120106e-08
16	0.479426	0.000000e+00	0.000000e+00
32	0.479426	0.000000e+00	0.000000e+00
64	0.479426	0.000000e+00	0.000000e+00
128	0.479426	0.000000e+00	0.000000e+00
256	0.479426	0.000000e+00	0.000000e+00

Bloque: $x = 0.950000$			
n	$S_n(x)$	$\Delta_n(x)$	$e_a(x)$
2	0.807104	1.428958e-01	1.770476e-01
4	0.813414	6.309615e-03	7.756957e-03
8	0.813416	1.722631e-06	2.117775e-06
16	0.813416	1.221245e-15	1.501379e-15
32	0.813416	0.000000e+00	0.000000e+00
64	0.813416	0.000000e+00	0.000000e+00
128	0.813416	0.000000e+00	0.000000e+00
256	0.813416	0.000000e+00	0.000000e+00

Bloque: $x = 10.000000$			
n	$S_n(x)$	$\Delta_n(x)$	$e_a(x)$
2	-4.840972	8.557787e+00	1.767783e+00
4	-0.874116	3.966857e+00	4.538136e+00
8	-0.544034	3.300814e-01	6.067289e-01
16	-0.544021	1.332032e-05	2.448494e-05
32	-0.544021	0.000000e+00	0.000000e+00
64	-0.544021	0.000000e+00	0.000000e+00
128	-0.544021	0.000000e+00	0.000000e+00
256	-0.544021	0.000000e+00	0.000000e+00

Estos resultados demuestran que, aunque la serie de $\sin(x)$ tiene radio de convergencia infinito, su evaluación directa mediante series de Taylor puede volverse numéricamente inestable para valores grandes de x . Esto se debe a la presencia de términos alternantes de gran magnitud que

generan cancelaciones significativas, provocando pérdida de precisión en aritmética de punto flotante.

En particular, el caso $x=10$ ilustra perfectamente la fase preliminar de la inestabilidad numérica. Aunque la arquitectura logra converger exitosamente al valor correcto (-0.544021) en $n=16$, las iteraciones iniciales exhiben una oscilación violenta y un error de magnitud considerable (llegando a -4.84 en $n=2$). Este comportamiento transitorio comprueba que la mantisa está operando al límite de su capacidad para alinear exponentes. Este fenómeno es el precursor de una cancelación catastrófica inminente si el valor de x continuara creciendo, justificando de manera concluyente la necesidad de implementar técnicas de reducción de argumento para garantizar la robustez del sistema en cualquier escenario.

5. Arquitectura del Software y Diagramas

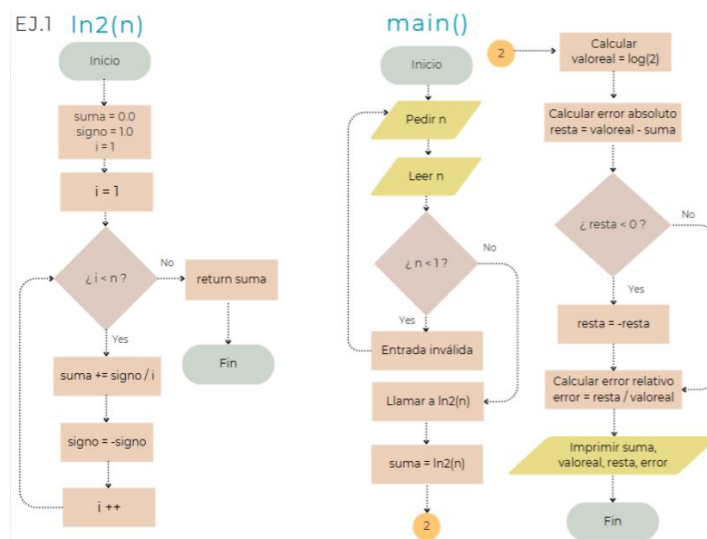
Para garantizar la documentación rigurosa de la práctica, se exponen los diagramas de flujo individuales de las 33 series matemáticas implementadas en el nivel 3. Esta representación visual detalla la mecánica interna de cada algoritmo, evidenciando el uso eficiente de las estructuras de control iterativas y la correcta validación de los dominios matemáticos.

Asimismo, esta vista de bajo nivel se complementa con la topología estructural del software a través de los grafos de llamadas (*Call Graphs*) autogenerados por Doxygen. Estos grafos demuestran la interdependencia modular de nuestra arquitectura, ilustrando, por ejemplo, la invocación de generadores de memoria dinámica (Bernoulli/Euler) por parte de las funciones trigonométricas e hiperbólicas de alto nivel.

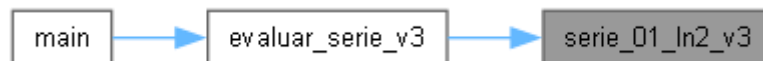
i. Serie 01: $\ln(2)$

Debido a su naturaleza trascendental, el logaritmo natural de 2 aparece constantemente al estudiar procesos de decaimiento; es el factor clave para realizar la conversión entre la constante de desintegración y el periodo de semidesintegración.

Diagrama de flujo en complejidad $O(n)$.



Grafo de llamadas de Doxygen

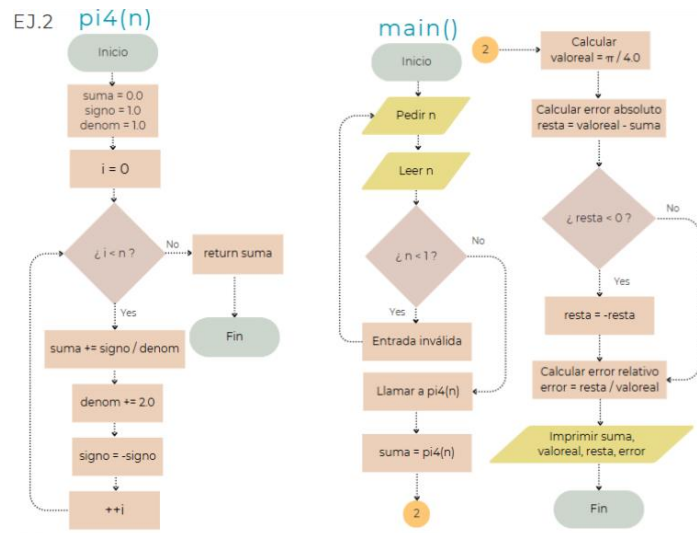


El Nivel 1 resolvía el signo alternante $(-1)^k$ utilizando costosos ciclos anidados y operaciones dependientes de `math.h`, generando un severo cuello de botella. En el Nivel 3, la complejidad se redujo a $O(n)$ global mediante un **patrón de estado conmutativo** (ejecutando simplemente `signo = -signo` por ciclo) y un incremento lineal del divisor. Esta refactorización permite iterar cientos de veces sin agotar los ciclos del procesador, estabilizando la lentitud natural de la serie.

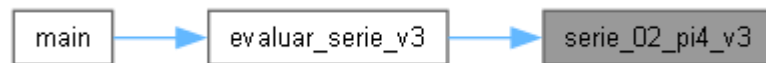
ii. Serie 02: $\frac{\pi}{4}$

Se realiza mediante la llamada serie de Leibniz, la cual fue creada por Gottfried Wilhelm Leibniz. Esta serie es una de las formas más conocidas para representar el número π a través de una suma infinita alternante. Teniendo la información anterior, sabemos que el valor es: 0.785398163397...

Diagrama de Flujo en complejidad $O(n)$



Grafo de llamadas de Doxygen



Análisis: El Nivel 3 es más eficiente que el Nivel 1 porque evita recalculando cada término desde cero y aprovecha una lógica incremental. Mientras que en el Nivel 1 se repiten operaciones innecesarias, aumentando la complejidad a $O(n^2)$, en el Nivel 3 el signo y el denominador se actualizan directamente en cada iteración ($\text{signo} = -\text{signo}$, $\text{denom} += 2$), reduciendo la complejidad a $O(n)$.

Esto no solo mejora el tiempo de ejecución, sino que también reduce la acumulación de errores numéricos, logrando una implementación más rápida y estable.

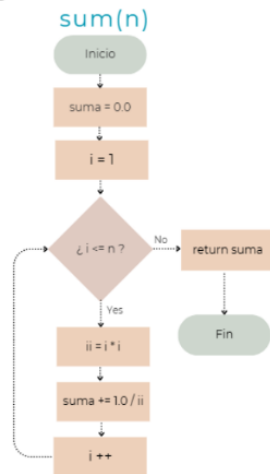
iii. Serie 03: $\frac{\pi^2}{6}$

Fue un desafío matemático importante durante décadas, que buscaba la suma exacta de la serie $\ln(2)$.

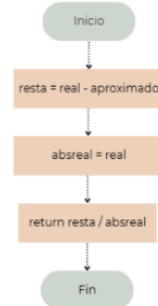
Este es el valor exacto al que converge la serie infinita conocida como el problema de Basilea, formulado por Pietro Mengoli en 1644 y resuelto por Leonhard Euler en 1735. Esta suma representa la suma de los inversos de los cuadrados de los números naturales. Teniendo la información anterior, sabemos que el valor es: 1.644934...

Diagrama de Flujo en complejidad $O(n)$

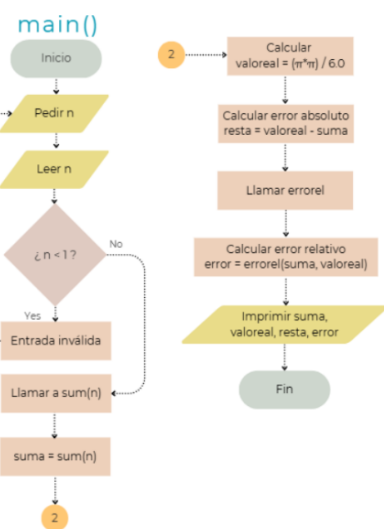
EJ.3



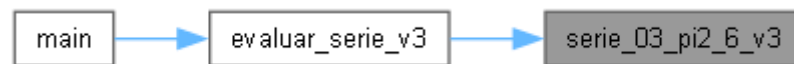
**errorel(aproximado,
real)**



EJ.3



Grafo de llamadas de Doxygen

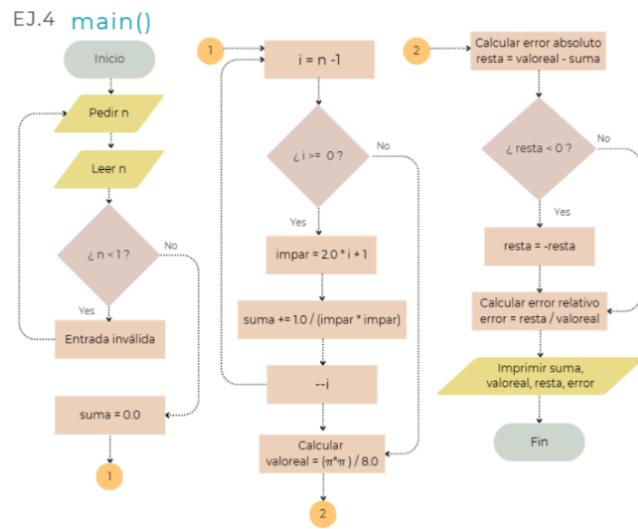


Análisis: El Nivel 3 mejora la implementación del Nivel 1 al evitar cálculos redundantes y organizar el algoritmo de forma modular. En lugar de recalculer operaciones innecesarias en cada iteración, se utiliza un ciclo simple que acumula directamente $\frac{1}{i^2}$, logrando una complejidad ($O(n)$). Además, al realizar los cálculos en tipo *double* y separar funciones como la suma y el error, se obtiene una mayor precisión numérica y un código más claro, eficiente y reutilizable.

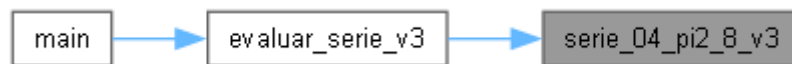
iv. Serie 04: $\frac{\pi^2}{8}$

Es un valor constante notable en matemáticas, que surge frecuentemente en series infinitas, análisis de Fourier y teoría de números. Su aparición más destacada es en la suma de los inversos de los cuadrados impares. Teniendo la información anterior, sabemos que el valor es: 1.2337...

Diagrama de Flujo en complejidad $O(n)$



Grafo de llamadas de Doxygen



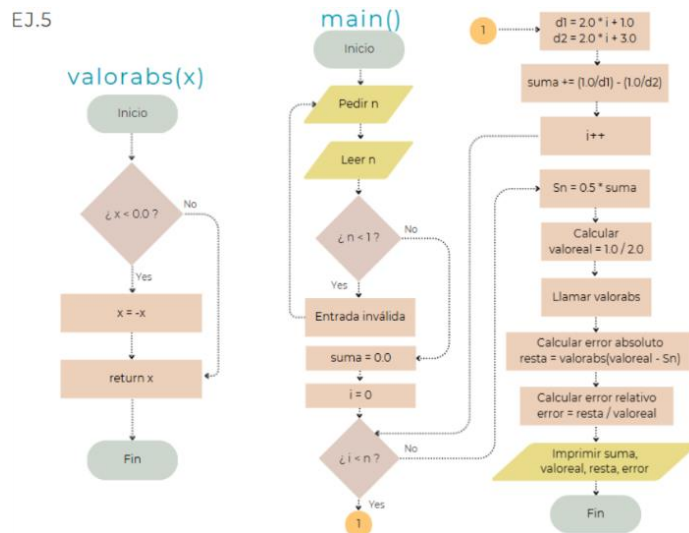
Análisis: El Nivel 3 mejora al Nivel 1 al optimizar tanto la eficiencia como la precisión numérica. En lugar de recalculer cada término de forma directa, se utiliza un ciclo simple en $O(n)$ y se aplica una estrategia de suma en orden inverso, comenzando por los términos más pequeños. Esto reduce el error de redondeo acumulado y mejora la estabilidad del resultado. En conjunto, se obtiene una implementación más rápida y numéricamente más precisa.

v. **Serie 05:** $\frac{1}{2}$

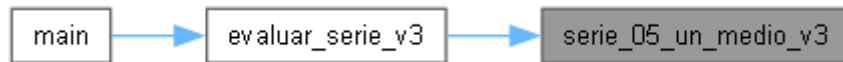
Para expresar el valor de cantidades que no contienen un número exacto de veces la unidad elegida, o de cantidades menores que la unidad, se han creado las fracciones comunes, que se lee un medio y significa la mitad de algo y es la razón común de muchas series geométricas convergentes. Esta serie es famosa por la Paradoja de Zenón, donde para llegar a un destino siempre debes recorrer primero la mitad de la distancia restante. Teniendo la información anterior, sabemos que el valor es: 0.5

Diagrama de Flujo en complejidad $O(n)$

EJ.5



Grafo de llamadas de Doxygen

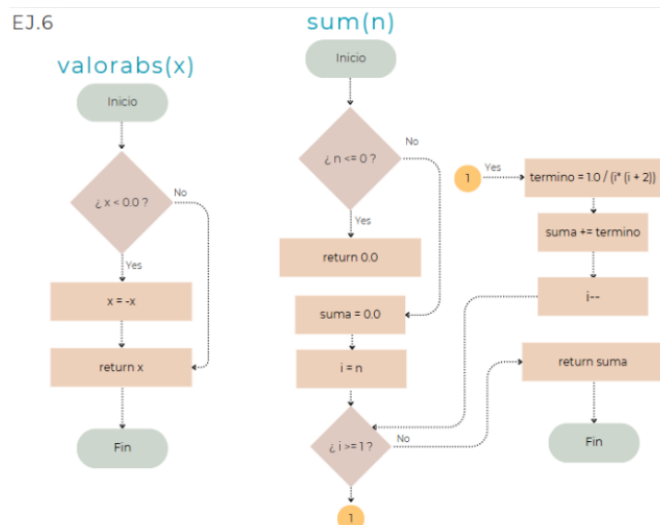


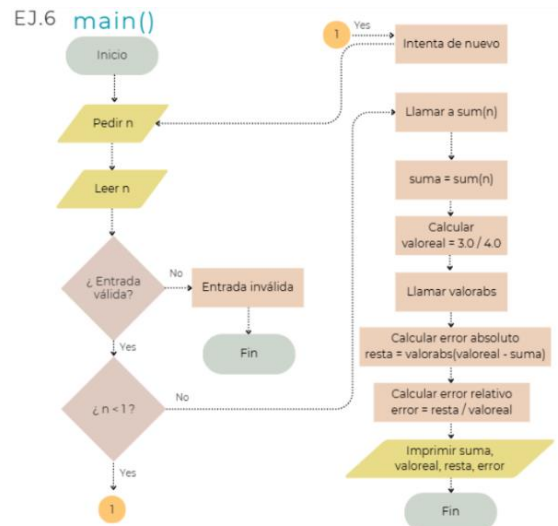
Análisis: El Nivel 3 mejora significativamente al Nivel 1 al aprovechar la forma telescópica de la serie, evitando cálculos redundantes. Mientras que en el Nivel 1 se evalúan directamente fracciones más complejas en cada iteración, aquí se simplifica cada término como una resta de fracciones, lo que permite cancelaciones sucesivas entre términos. Esto reduce el número de operaciones, mejora la eficiencia a $O(n)$ y aumenta la precisión numérica al minimizar errores de redondeo.

vi. Serie 06: $\frac{3}{4}$

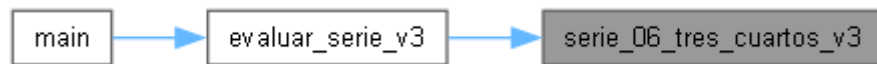
Podemos expresarlo como la suma de mitades sucesivas (como en la paradoja de Zenón) pero empezando desde un punto distinto o multiplicando la serie base por una constante. Para resolver esta serie, utilizamos el método de fracciones parciales. Teniendo la información anterior, sabemos que el valor es: 0.75

Diagrama de Flujo en complejidad $O(n)$





Grafo de llamadas de Doxygen

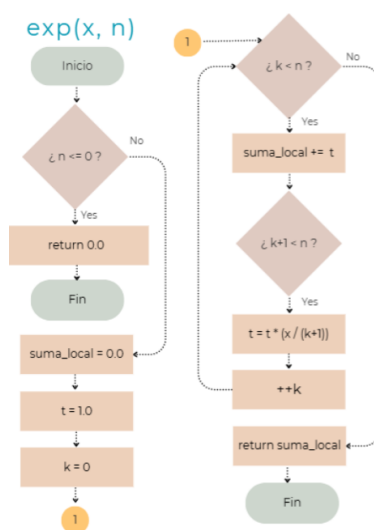


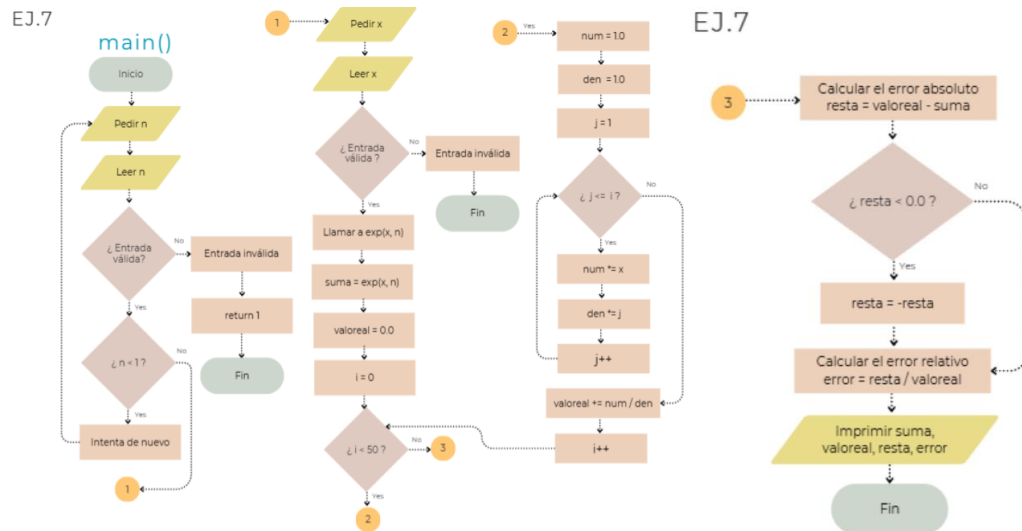
Análisis: El Nivel 3 mejora al Nivel 1 al aprovechar la forma telescópica de la serie y optimizar el orden de suma. En lugar de realizar cálculos redundantes en cada término, se utiliza una suma directa en $O(n)$ y se acumulan primero los términos más pequeños mediante suma inversa, lo que reduce el error de redondeo. Esto da como resultado una implementación más eficiente y numéricamente más estable.

vii. Serie 07: e^x

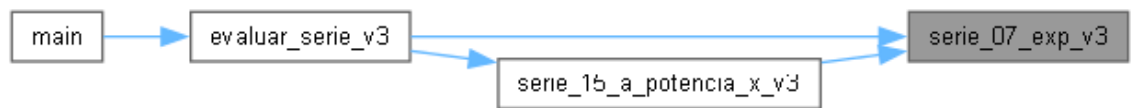
Se representa matemáticamente mediante la serie de potencias infinita, conocida también como serie de Taylor Maclaurin, válida para todo número real x . Teniendo la información anterior, sabemos que el valor depende del valor de x .

Diagrama de Flujo en complejidad $O(n)$





Grafo de llamadas de Doxygen

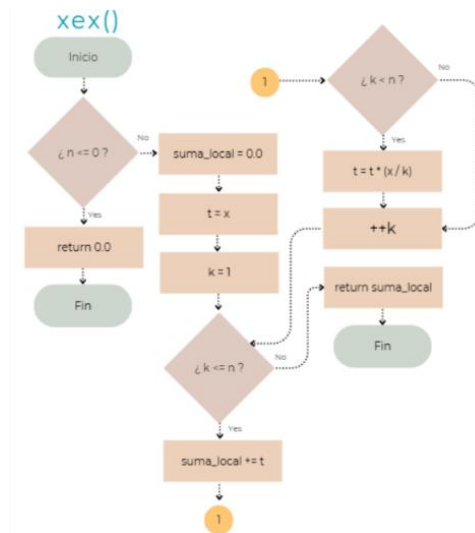


Análisis: El Nivel 3 mejora significativamente al Nivel 1 al utilizar una relación de recurrencia para generar cada término a partir del anterior, evitando el cálculo repetido de potencias y factoriales. Mientras que el Nivel 1 emplea ciclos anidados con complejidad ($O(n^2)$), esta implementación reduce el costo a ($O(n)$) al usar $t_{k+1} = t_k \cdot \frac{x}{k+1}$. Esto no solo mejora el rendimiento, sino que también reduce errores numéricos, logrando una aproximación más eficiente y estable.

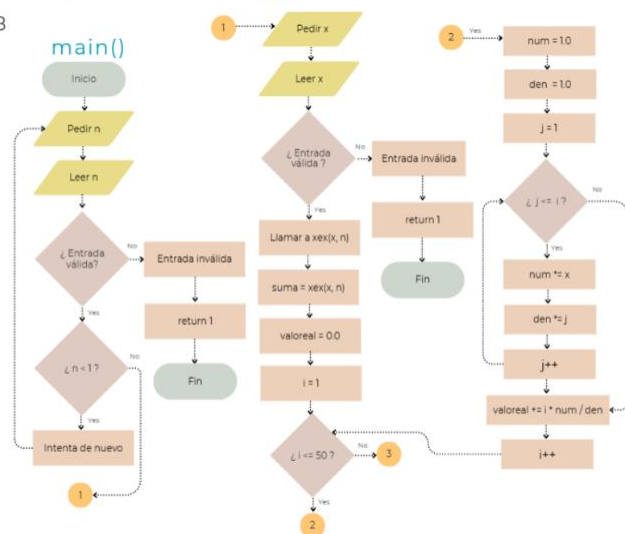
viii. Serie 08: xe^x

Contexto matemático: La función $f(x) = xe^x$ es la función que define a la Función W de Lambert, denotada como $f(x)$. Esta función no puede expresarse en términos de funciones elementales (como senos, logaritmos o raíces), por lo que xe^x es la clave para resolver ecuaciones que de otro modo serían imposibles de despejar. Teniendo la información anterior, sabemos que el valor es: Depende del valor de x .

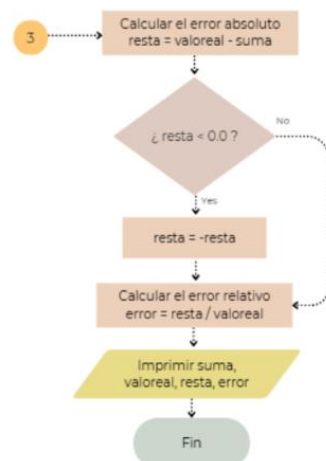
Diagrama de Flujo en complejidad $O(n)$



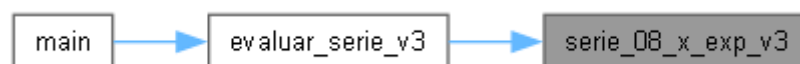
EJ.8



EJ.8



Grafo de llamadas de Doxygen

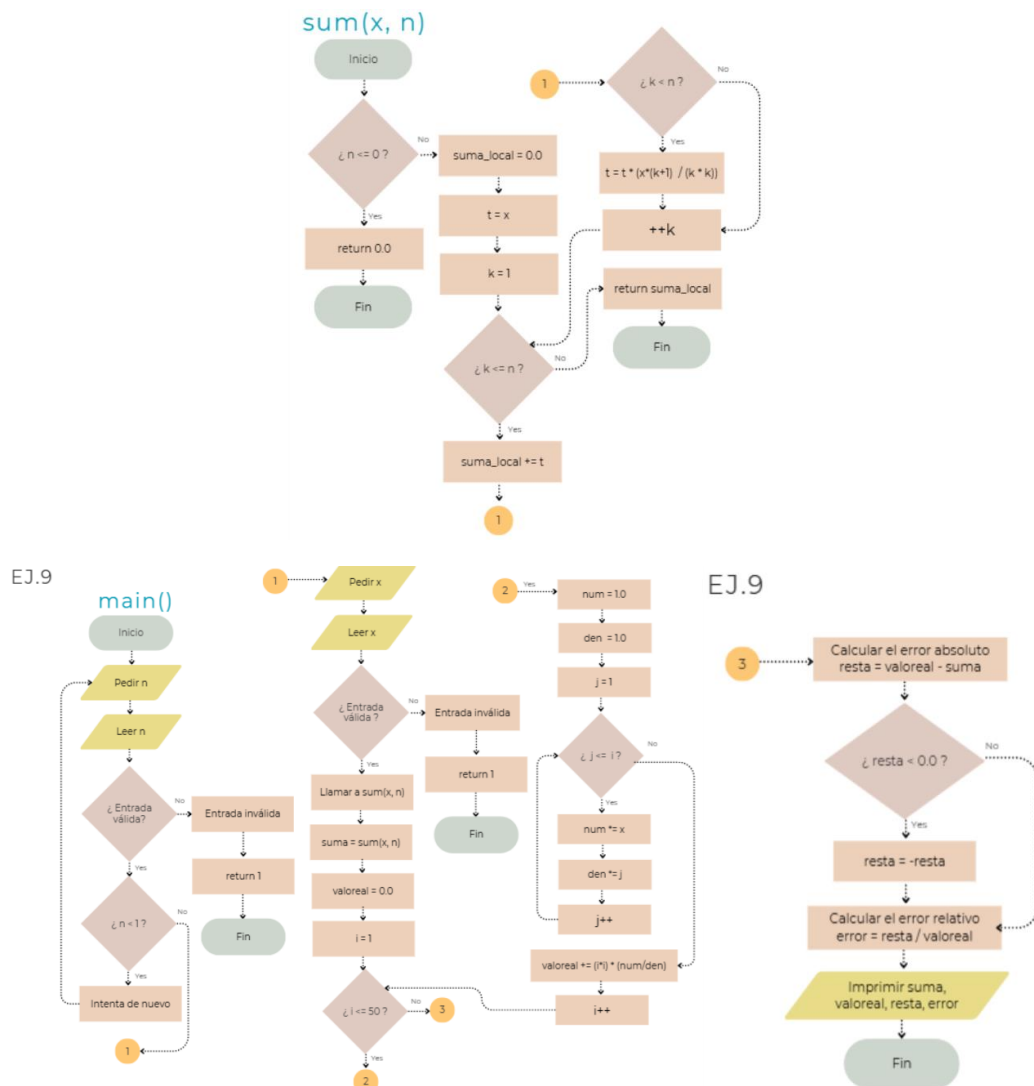


Análisis: El Nivel 3 mejora significativamente al Nivel 1 al utilizar una relación de recurrencia para generar cada término a partir del anterior, evitando el cálculo repetido de potencias y factoriales. Mientras que el Nivel 1 emplea ciclos anidados con complejidad ($O(n^2)$), esta implementación reduce el costo a ($O(n)$) al usar la relación $t_{k+1} = t_k \cdot \frac{x}{k}$. Esto no solo mejora el rendimiento, sino que también reduce errores numéricos, logrando una aproximación más eficiente y estable.

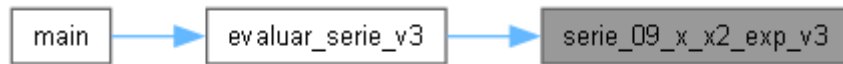
ix. Serie 09: $(x + x^2)e^x$

Los coeficientes que acompañan a en este tipo de funciones (en este caso el polinomio $(x+x^2+2x)$) están relacionados con los Números de Stirling de segunda especie, que cuentan de cuántas maneras se puede particionar un conjunto de n elementos. Para n^2 , los coeficientes del polinomio resultante (1 y 1) corresponden a los valores de la tabla de Stirling para ese orden. Teniendo la información anterior, sabemos que el valor es: Depende del valor de x .

Diagrama de Flujo en complejidad $O(n)$



Grafo de llamadas de Doxygen

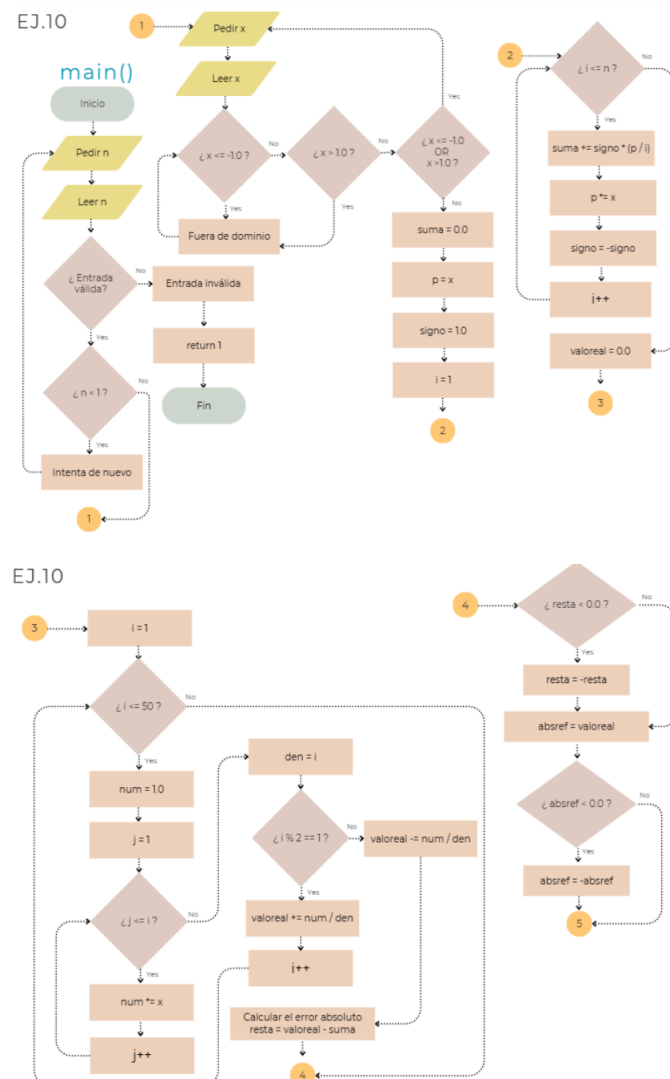


Análisis: El Nivel 3 mejora significativamente al Nivel 1 al utilizar una relación de recurrencia para generar cada término a partir del anterior, evitando el cálculo repetido de potencias y factoriales. Mientras que el Nivel 1 emplea ciclos anidados con complejidad ($O(n^2)$), esta implementación reduce el costo a ($O(n)$) al usar la relación $t_{k+1} = t_k \cdot \frac{x(k+1)}{k^2}$. Esto no solo mejora el rendimiento, sino que también reduce errores numéricos, logrando una aproximación más eficiente y estable.

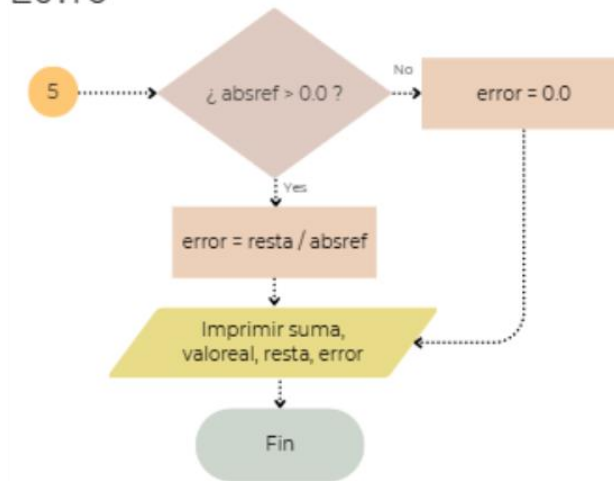
x. Serie 10: $\ln(1+x)$

La representación de $\ln(1+x)$ como serie de potencias se conoce como la Serie de Mercator (por Nicholas Mercator, quien la publicó en 1668) [12]. A diferencia de xe^x (que converge para todo x), esta serie solo es válida para $-1 < x \leq 1$. Teniendo la información anterior, sabemos que el valor de $\ln(1+x)$ es: Depende del valor de x en el rango de $-1 < x \leq 1$.

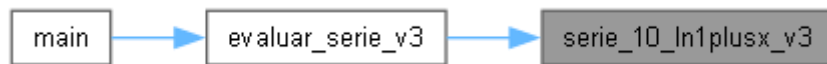
Diagrama de Flujo en complejidad $O(n)$



EJ.10



Grafo de llamadas de Doxygen



Análisis: El Nivel 3 mejora significativamente al Nivel 1 al utilizar una relación de recurrencia para generar cada término a partir del anterior, evitando el cálculo repetido de potencias. Mientras que el Nivel 1 emplea ciclos anidados con complejidad ($O(n^2)$), esta implementación reduce el costo a ($O(n)$) al usar la actualización $p_{i+1} = p_i \cdot x$ y el cambio de signo ($\text{signo} = -\text{signo}$). Esto no solo mejora el rendimiento, sino que también reduce errores numéricos, logrando una aproximación más eficiente y estable.

xi. Serie 11: $\frac{1}{2} \ln \left(\frac{1+x}{1-x} \right)$

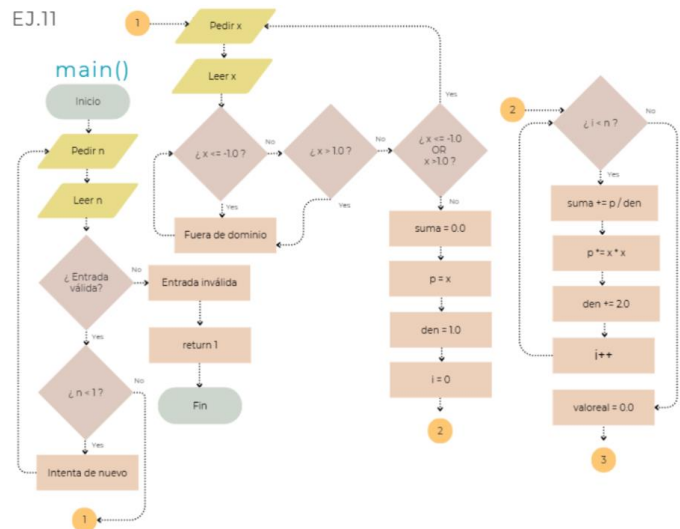
Esta serie se obtiene combinando dos series de Taylor fundamentales:

$$\ln(1+x): x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots$$

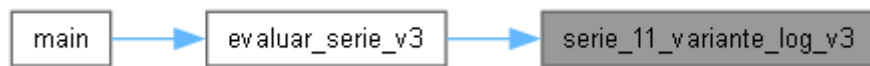
$$\ln(1-x): -x - \frac{x^2}{2} - \frac{x^3}{3} - \frac{x^4}{4} - \dots$$

Al restar la segunda de la primera $\ln(1+x) - \ln(1-x)$, los términos con potencias pares se cancelan y los términos con potencias impares se duplican. Usando la propiedad de los logaritmos $\ln(1+x) - \ln(1-x) = \frac{1}{2} \ln \left(\frac{1+x}{1-x} \right)$. Teniendo la información anterior, sabemos que el valor es: Depende del valor de x en el rango de $-1 < x \leq 1$.

Diagrama de Flujo en complejidad $O(n)$



Grafo de llamadas de Doxygen

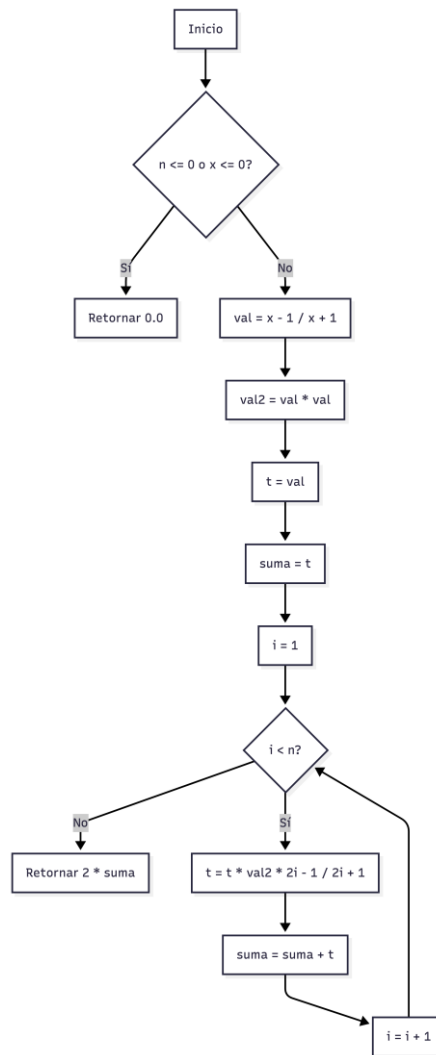


Análisis: El Nivel 3 mejora significativamente al Nivel 1 al utilizar una relación de recurrencia para generar las potencias impares de forma eficiente, evitando recalcular x^{2k+1} en cada iteración. Mientras que el Nivel 1 emplea ciclos anidados con complejidad $(O(n^2))$, esta implementación reduce el costo a $(O(n))$ al usar la actualización $p_{k+1} = p_k \cdot x^2$. Esto no solo mejora el rendimiento, sino que también reduce errores numéricos, logrando una aproximación más eficiente y estable.

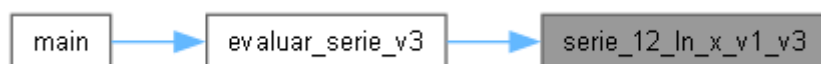
xii. Serie 12: $\ln(x)$

La función $\ln(x)$, conocida como logaritmo natural, es una herramienta fundamental en el cálculo y las ciencias aplicadas por su relación directa con la función exponencial y su capacidad para modelar fenómenos de crecimiento, decaimiento y análisis de datos; definida únicamente para $(x > 0)$, cumple propiedades esenciales que facilitan su manipulación matemática y la vuelven clave en procesos como la derivación e integración. Para su aproximación numérica, puede expresarse mediante una serie de Taylor alrededor de $(x = 1)$, lo que permite representarla como una suma infinita de términos alternantes, siendo especialmente útil cuando los valores se encuentran cercanos a este punto.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen



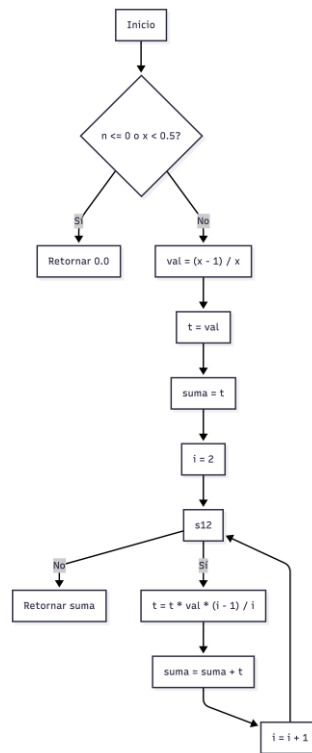
La versión de nivel 3 es superior porque elimina el cálculo redundante de potencias dentro de un ciclo interno, reduciendo la complejidad computacional de $O(n^2)$ a $O(n)$. En lugar de recalculer cada término desde cero, reutiliza el término anterior mediante una relación recursiva, lo que mejora significativamente la eficiencia y disminuye el error de redondeo acumulado. Además, incluye validación de dominio $((x > 0) \text{ y } (n > 0))$, lo que la hace más robusta y segura frente a entradas inválidas.

xiii. Serie 13: $\ln(x)$

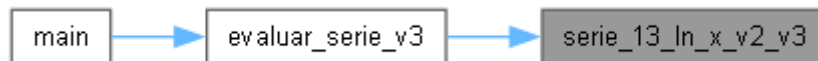
Puede representarse mediante formas alternativas de series que mejoran su estabilidad numérica dependiendo del intervalo de evaluación, como aquellas basadas en transformaciones tipo $\frac{x-1}{x+1}$ o $\frac{x-1}{x}$, las cuales permiten expresar el logaritmo como sumas de potencias con coeficientes racionales y comportamiento más controlado. Estas formulaciones son especialmente útiles

porque amplían el rango práctico de convergencia y reducen errores cuando (x) no está tan cercano a 1, ofreciendo mayor eficiencia en cálculos computacionales.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

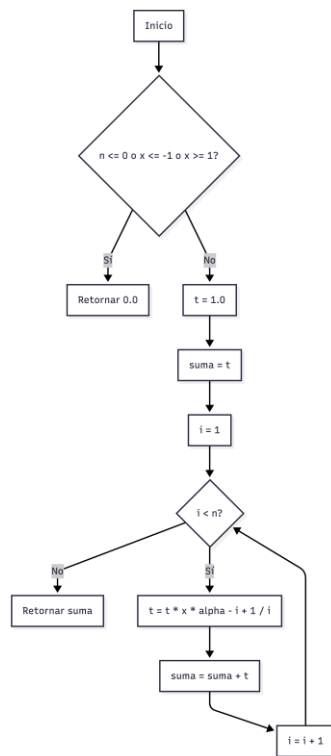


La versión de nivel 3 mejora claramente a la de nivel 1 porque evita recalcular potencias en cada iteración, eliminando el ciclo interno y reduciendo la complejidad de $(O(n^2))$ a $(O(n))$. En lugar de construir cada término desde cero, utiliza una relación recursiva que genera el siguiente término a partir del anterior, lo que hace el algoritmo más eficiente y numéricamente más estable. Además, incorpora una validación básica del dominio $((n > 0) \text{ y } (x) \text{ en un rango adecuado})$, aumentando la robustez del método frente a entradas inválidas.

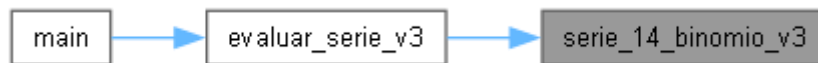
xiv. Serie 14: $(1 + x)^\alpha$

Conocida como el binomio generalizado, es fundamental en el análisis matemático y en métodos numéricos porque extiende la potencia a exponentes reales o incluso complejos, permitiendo modelar una gran variedad de fenómenos; su desarrollo en serie de Taylor (o binomial) alrededor de $x = 0$ la expresa como una suma infinita de términos con coeficientes que dependen de α , lo que facilita su aproximación sin necesidad de calcular potencias directamente. Esta serie es especialmente útil cuando x es pequeño, pero su convergencia está restringida a $|x| < 1$ en el caso general, y puede volverse ineficiente o numéricamente inestable si x se acerca a los límites del intervalo o si α toma valores que generan coeficientes grandes, implicando un mayor costo computacional y posible acumulación de errores.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

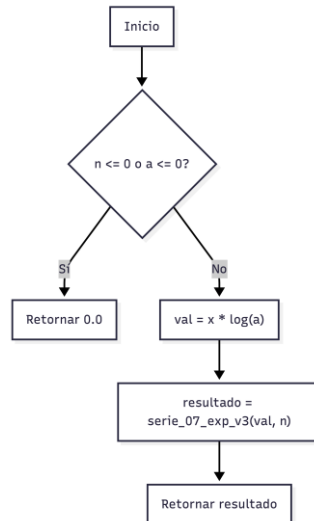


La versión de nivel 3 es mejor porque transforma el cálculo del binomio en un proceso iterativo eficiente, evitando el doble ciclo que recalcula coeficientes y potencias en cada término. Al generar cada término a partir del anterior, reduce la complejidad de $O(n^2)$ a $O(n)$, disminuye el costo computacional y mejora la estabilidad numérica al evitar multiplicaciones repetitivas innecesarias. Además, incluye una validación del dominio $((-1 < x < 1) \text{ y } (n > 0))$, lo que asegura la convergencia de la serie y hace el algoritmo más robusto.

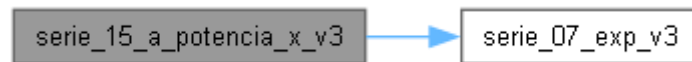
xv. Serie 15: a^x

Conocida como función exponencial de base arbitraria, es fundamental en matemáticas, física e ingeniería por su capacidad de modelar crecimiento y decaimiento continuo; puede expresarse en términos de la exponencial natural como $a^x = e^{x \ln(a)}$, lo que permite desarrollar su representación en serie de Taylor alrededor de $x = 0$ como una suma infinita de términos en potencias de x . Esta expansión resulta especialmente útil en métodos numéricos y computacionales para aproximar su valor sin calcular potencias directamente, manteniendo buena precisión cuando x es cercano a cero.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

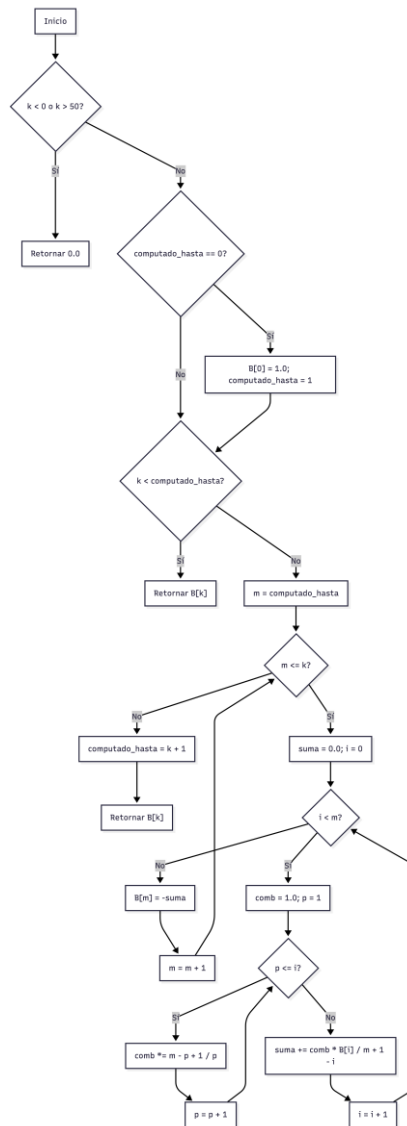


La versión de nivel 3 es más robusta que la de nivel 1 porque, aunque mantiene la misma idea eficiente de reutilizar la serie de (e^x) mediante la identidad $(a^x = e^{x \ln(a)})$, ahora incorpora validación de entradas, evitando valores no definidos como $a \leq 0$ o $n \leq 0$. Además, al apoyarse en una versión optimizada de la exponencial (nivel 3), hereda sus mejoras en eficiencia y estabilidad numérica, logrando un método más seguro y consistente sin sacrificar rendimiento.

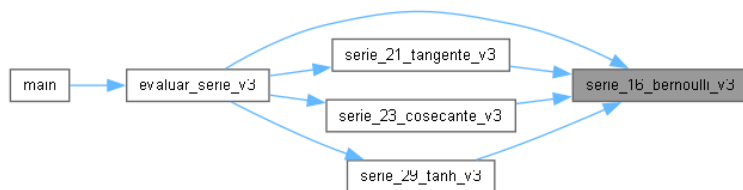
xvi. Serie 16: B_k

Conocidos como número de Bernoulli, son constantes fundamentales en el análisis matemático que aparecen de forma recurrente en expansiones en serie de funciones especiales, sumas de potencias y métodos numéricos avanzados; su importancia radica en que permiten expresar de manera compacta desarrollos como los de funciones trigonométricas, exponenciales y logarítmicas, influyendo directamente en la precisión y estructura de dichas series.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

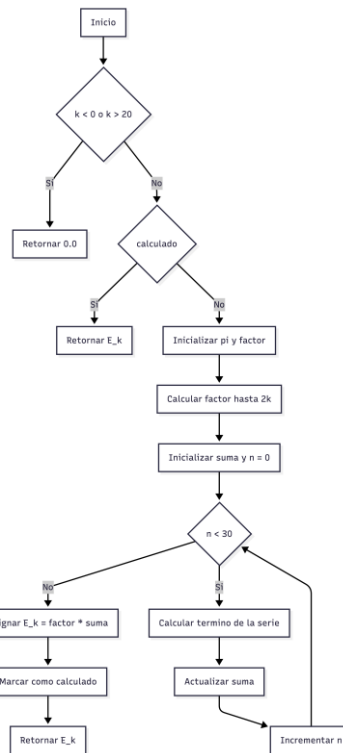


La versión de nivel 3 es claramente superior porque sustituye la recursión costosa del nivel 1 por un enfoque de programación dinámica con memoización, evitando recalcular repetidamente los mismos números de Bernoulli. Mientras que la versión original tiene una complejidad muy alta debido a llamadas recursivas y cálculos redundantes de factoriales, la versión optimizada almacena resultados en memoria y los reutiliza en tiempo constante cuando ya fueron calculados, reduciendo drásticamente el costo computacional. Además, controla el rango de entrada ($(0 \leq k \leq 50)$), lo que previene desbordamientos y hace el algoritmo más estable y eficiente.

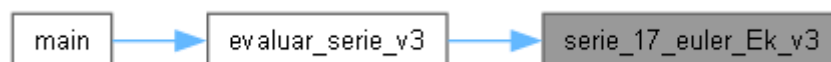
xvii. Serie 17: E_k

Conocidos como números de Euler, son constantes matemáticas que aparecen en el desarrollo en serie de funciones especiales, particularmente en expansiones relacionadas con funciones trigonométricas y hiperbólicas, desempeñando un papel clave en aproximaciones numéricas y análisis avanzado. Su estructura alternante y su rápido crecimiento en magnitud los hacen útiles para describir comportamientos complejos de ciertas series, pero también introducen dificultades computacionales, ya que pueden provocar inestabilidad numérica y acumulación de error cuando se utilizan en órdenes altos o en implementaciones con precisión limitada.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen



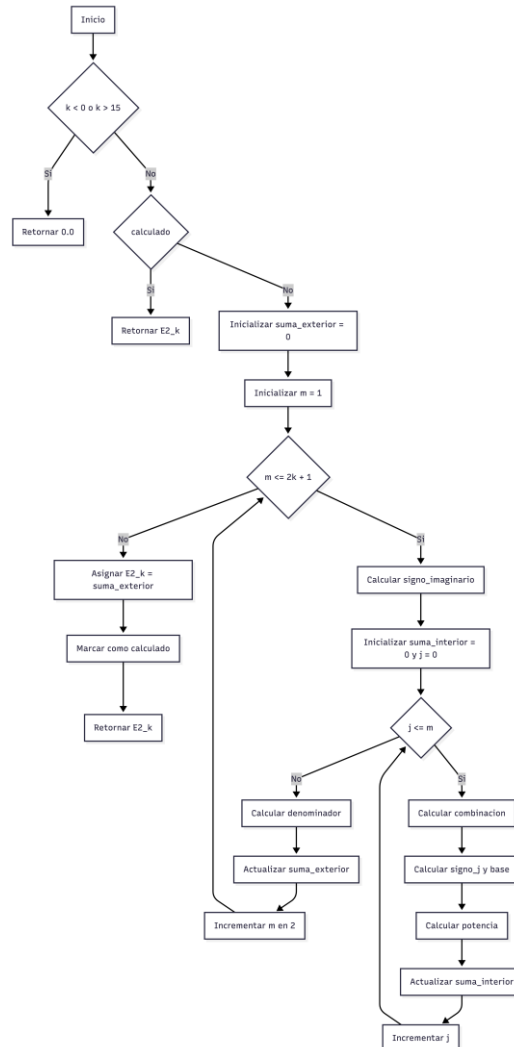
Optimiza tanto el cálculo como el uso de memoria: introduce memoización, evitando recalcular valores de (E_k) ya obtenidos, lo que reduce significativamente el tiempo en llamadas repetidas. Además, simplifica el cálculo del factor externo usando una forma iterativa más eficiente en lugar de múltiples productos independientes, disminuyendo operaciones redundantes. También reduce el número de términos en la suma infinita (de 50 a 30) manteniendo buena precisión, lo que mejora el rendimiento. Finalmente, incluye validación del rango de entrada $(0 \leq k \leq 20)$, previniendo errores numéricos y haciendo el algoritmo más estable.

xviii. Serie 18: E_{2k}

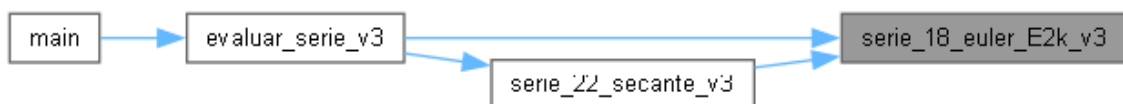
Conocidos como los números de Euler de índice par, son una subsecuencia de los números de Euler que aparecen específicamente en el desarrollo en serie de funciones como la secante y la tangente hiperbólica, siendo fundamentales en el análisis de aproximaciones numéricas y en la

descripción de comportamientos simétricos en ciertas funciones; su relevancia radica en que solo los índices pares producen valores distintos de cero en estos contextos, lo que simplifica algunas expresiones.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

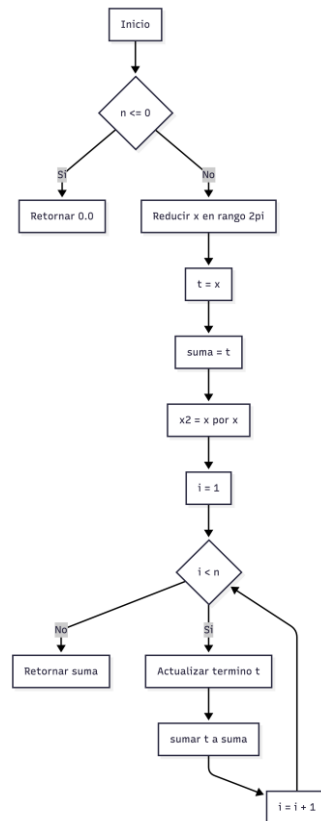


La versión de nivel 3 es superior porque elimina operaciones innecesarias y reduce el costo computacional al recorrer únicamente valores impares de m incrementando de 2 en 2, en lugar de evaluar y descartar los pares en cada iteración. Además, sustituye el cálculo repetitivo de factoriales por una construcción iterativa de combinaciones más eficiente, disminuyendo la complejidad interna. La incorporación de memoización permite reutilizar resultados previamente calculados en tiempo constante, lo que mejora mucho el rendimiento en llamadas repetidas. Finalmente, incluye control de rango ($(0 \leq k \leq 15)$), aumentando la estabilidad numérica y evitando desbordamientos.

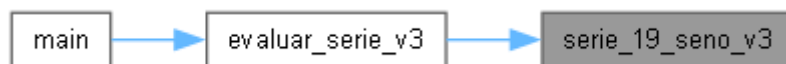
xix. Serie 19: $\text{sen}(x)$

La función $\text{sen}(x)$, conocida como seno, es una función trigonométrica fundamental en el estudio de fenómenos periódicos como ondas, oscilaciones y señales, siendo ampliamente utilizada en física, ingeniería y procesamiento de datos; su comportamiento suave y acotado permite representarla mediante una serie de Taylor alrededor de $(x = 0)$, expresándola como una suma infinita de potencias impares con signos alternantes, lo que facilita su aproximación numérica con alta precisión cuando (x) es cercano a cero.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

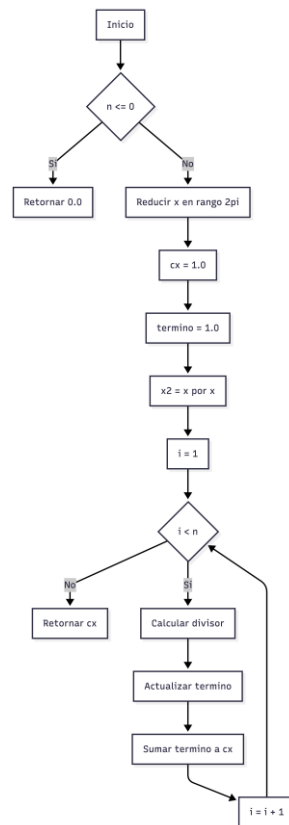


La versión de nivel 3 es mucho más eficiente y estable que la de nivel 1 porque elimina el cálculo repetitivo de potencias y factoriales en cada iteración, reduciendo la complejidad de $O(n^2)$ a $O(n)$. En lugar de recomputar cada término desde cero, utiliza una relación de recurrencia que genera directamente el siguiente término a partir del anterior, lo que disminuye operaciones y error numérico. Además, normaliza x usando fmod al rango $([0, \frac{2}{\pi}])$, mejorando la convergencia de la serie para valores grandes. Todo esto hace que el algoritmo sea más rápido, preciso y robusto.

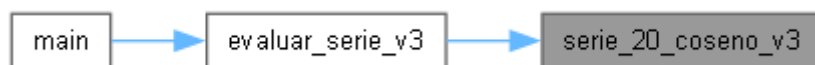
xx. Serie 20: $\cos(x)$

La función $\cos(x)$, conocida como coseno, es una función trigonométrica esencial en el análisis de fenómenos periódicos como ondas, vibraciones y señales, siendo ampliamente utilizada en física e ingeniería; su comportamiento suave y acotado permite representarla mediante una serie de Taylor alrededor de $(x = 0)$, expresándola como una suma infinita de potencias pares con signos alternantes, lo que facilita su aproximación numérica con alta precisión cuando (x) es cercano a cero. .

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

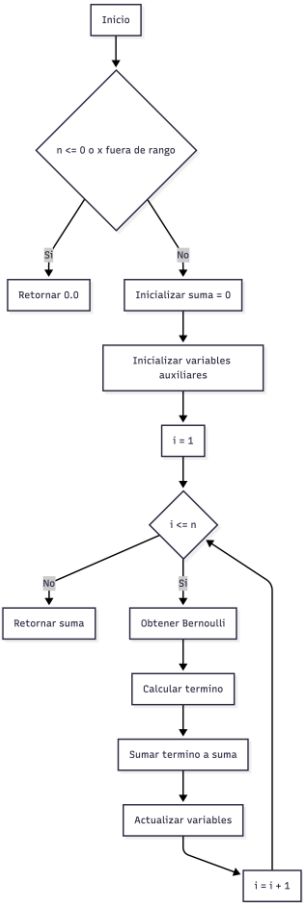


La versión de nivel 3 es mejor porque evita recalcular potencias y factoriales en cada iteración, eliminando el ciclo interno y reduciendo la complejidad de $O(n^2)$ a $O(n)$. Utiliza una relación de recurrencia para generar cada término a partir del anterior, lo que mejora significativamente la eficiencia y reduce el error de redondeo. Además, normaliza el valor de x con fmod para mantenerlo en un rango adecuado, favoreciendo la convergencia de la serie y haciendo el algoritmo más preciso y robusto.

xxi. **Serie 21:** $\tan(x)$

La función $\tan(x)$, conocida como tangente, es una función trigonométrica fundamental que describe relaciones de pendiente y aparece en el análisis de fenómenos periódicos, geometría y señales; puede representarse mediante una serie de Taylor alrededor de $(x = 0)$, expresándose como una suma infinita de potencias impares cuyos coeficientes están relacionados con los números de Bernoulli, lo que permite su aproximación numérica en valores cercanos a cero.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

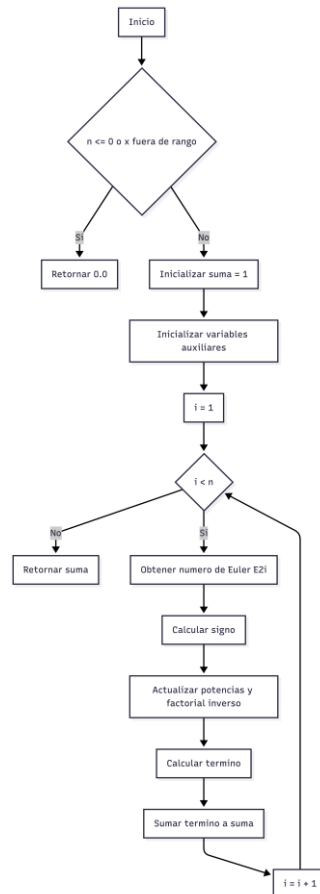


La versión de nivel 3 es muy superior porque elimina múltiples cálculos redundantes (potencias y factoriales) dentro de cada iteración, reduciendo la complejidad de $O(n^2)$ a $O(n)$. En lugar de recalcular todo desde cero, actualiza cada término de forma incremental mediante recurrencias, lo que mejora notablemente la eficiencia y la estabilidad numérica. Además, aprovecha la versión optimizada de los números de Bernoulli con memoización, evitando recomputaciones costosas. Finalmente, incluye validación del dominio $(|x| < \frac{\pi}{2})$, asegurando la convergencia de la serie y haciendo el algoritmo más robusto.

xxii. **Serie 22:** $\sec(x)$

La función $\sec(x)$, definida como el recíproco del coseno, es una función trigonométrica relevante en el análisis de fenómenos periódicos y en ciertas formulaciones de la geometría y la física; su expansión en serie de Taylor alrededor de $(x = 0)$ permite representarla como una suma infinita de potencias pares cuyos coeficientes están relacionados con los números de Euler, lo que la hace útil en aproximaciones numéricas cuando x es pequeño.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

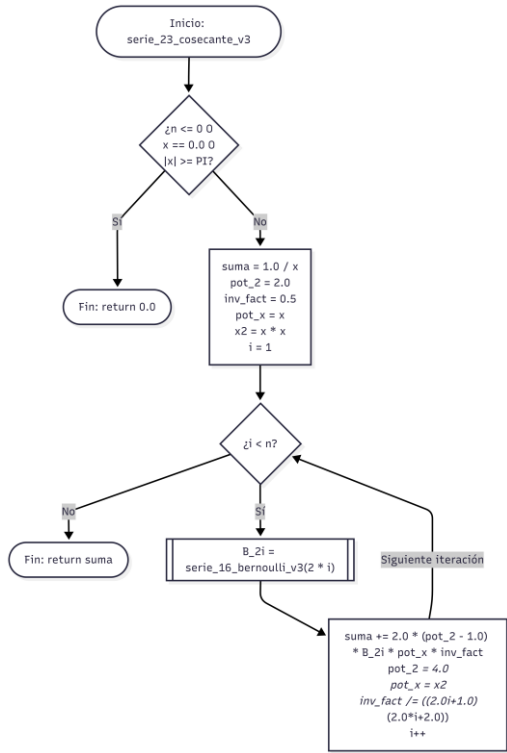


La versión de nivel 3 es mejor porque elimina el cálculo repetitivo de potencias y factoriales dentro de cada iteración, reduciendo la complejidad de $O(n^2)$ a $O(n)$. En lugar de recomputar x^{2n} y $(2n)!$ desde cero, los actualiza de forma incremental, lo que mejora la eficiencia y disminuye el error numérico. Además, utiliza la versión optimizada de los números de Euler con memoización, evitando cálculos redundantes. Finalmente, valida el dominio $(|x| < \frac{\pi}{2})$, garantizando la convergencia de la serie y haciendo el algoritmo más robusto.

xxiii. Serie 23: $\csc(x)$

Al ser el recíproco del seno, la cosecante es una función trigonométrica fundamental en el análisis de ondas, acústica y sistemas oscilatorios. Sin embargo, su expansión en serie de Maclaurin representa un desafío computacional de alto calibre debido a su estricta dependencia matemática de los números de Bernoulli para dictar su convergencia.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

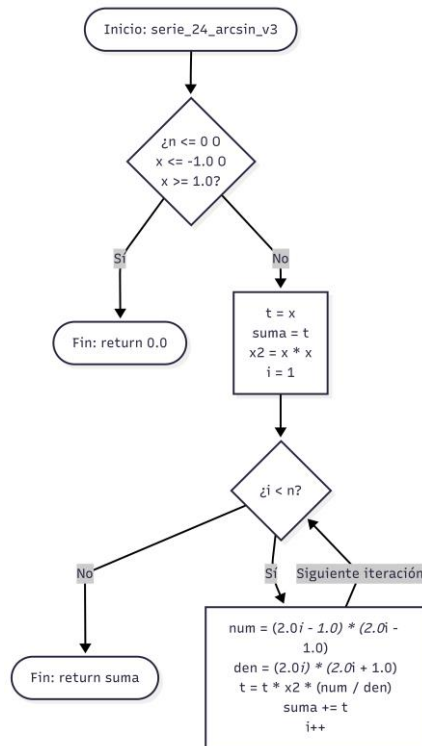


El Nivel 1 abordaba esta serie de manera ineficiente recalculando potencias y factoriales desde cero en cada iteración, lo que disparaba la complejidad temporal a $O(n^2)$. En el Nivel 3, la arquitectura se transformó en un modelo $O(n)$ actualizando recursivamente factores como `pot_2 *= 4.0` y `pot_x *= x2` y delegando el peso analítico a una subrutina dedicada para los números de Bernoulli. Esta refactorización elimina las operaciones anidadas redundantes, garantizando una ejecución milimétrica y estable.

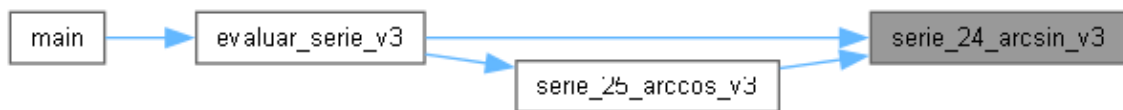
xxiv. Serie 24: $\arcsen(x)$

El arcoseno es indispensable en la resolución de problemas inversos en cinemática, trigonometría esférica y en la transformación de coordenadas rectangulares a polares. Su naturaleza matemática requiere una estricta validación del dominio de entrada, confinando la ejecución a valores donde la magnitud de x sea menor a 1 para asegurar la convergencia de la serie de Maclaurin y evitar desbordamientos aritméticos.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

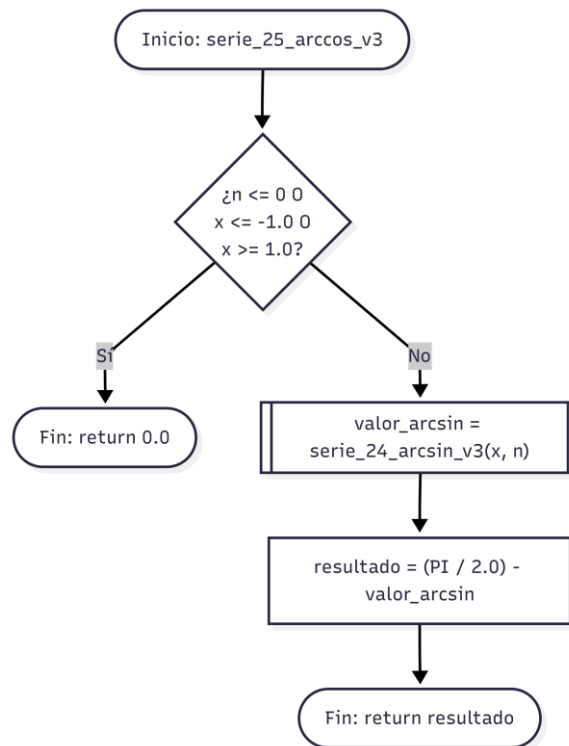


A diferencia de las implementaciones del Nivel 1 que calculaban dobles factoriales y potencias de forma repetitiva y aislada, este algoritmo de Nipor vel 3 alcanza una eficiencia temporal de $O(n)$ pura. Esto se logra mediante una recurrencia algebraica autosuficiente; cada iteración deriva su valor multiplicando el término anterior por un factor de actualización fraccionario exacto, eliminando el uso de librerías externas. Además, como evidencia el grafo arquitectónico, esta extrema eficiencia convierte a la función en un motor subyacente ideal, permitiendo que otras operaciones (como el $\arccos(x)$) deleguen su carga computacional hacia ella con total seguridad.

xxv. Serie 25: $\arccos(x)$

El $\arccos$ es una herramienta matemática vital para calcular ángulos espaciales entre vectores (vía producto punto), trayectorias orbitales y fases en el procesamiento de señales. Al igual que el arcoseno, su dominio de convergencia real está estrictamente acotado al intervalo $[-1, 1]$, requiriendo una validación temprana en el flujo de ejecución para evitar el colapso del sistema al intentar procesar ángulos imaginarios.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

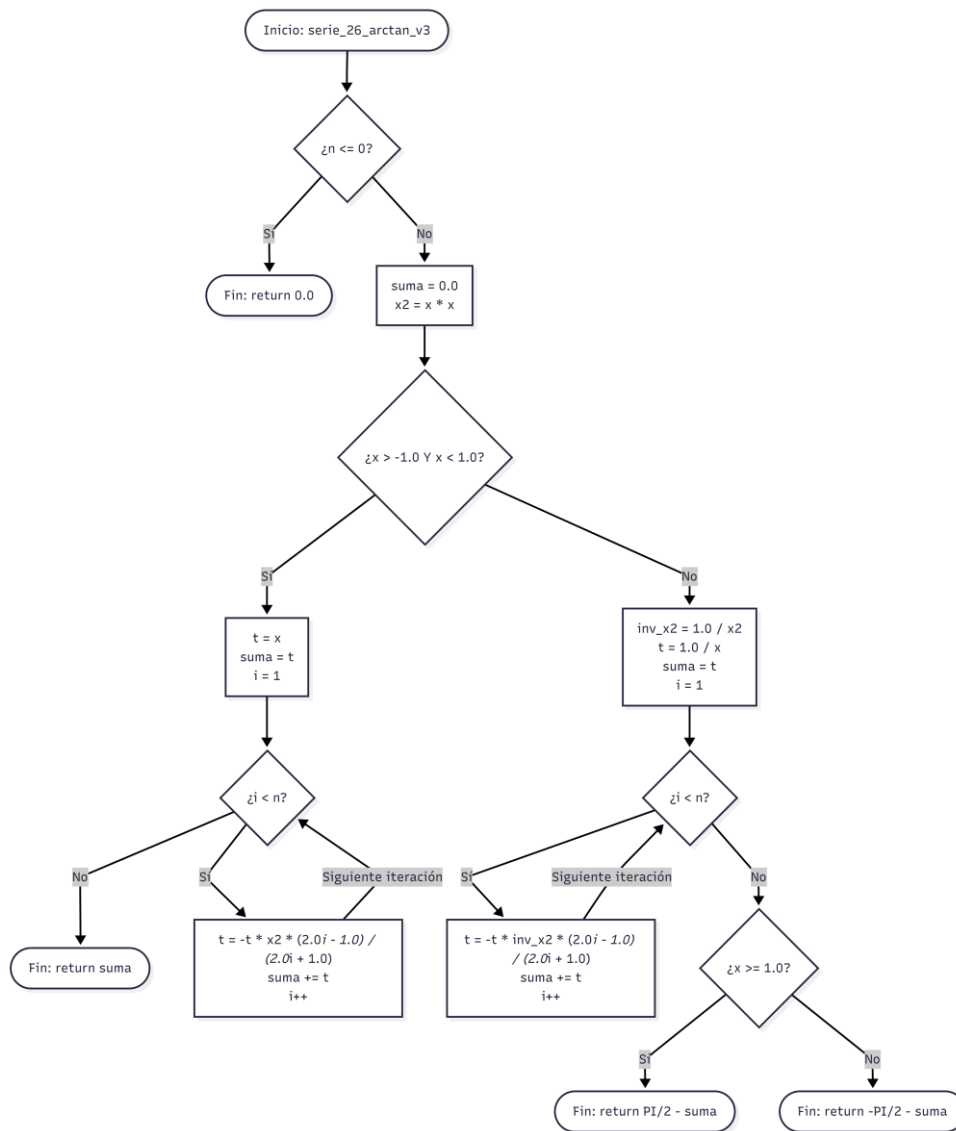


En lugar de codificar y procesar una nueva serie de Maclaurin desde cero, este diseño aplica la identidad trigonométrica fundamental: $\arccos(x) = \frac{\pi}{2} - \arcsin(x)$. El grafo de dependencias de Doxygen ilustra elegantemente cómo esta función actúa como un cliente ligero, delegando todo el peso computacional al arcoseno (Serie 24). El resultado es una función que hereda la complejidad $O(n)$ de su subrutina.

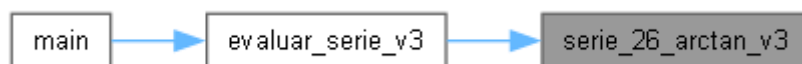
xxvi. Serie 26: $\arctan(x)$

La $\arctan$ es la espina dorsal de la trigonometría computacional, utilizada masivamente en la conversión de coordenadas cartesianas a polares, el cálculo de fases en números complejos y la navegación de sistemas autónomos. El reto algorítmico principal radica en su radio de convergencia natural; la serie estándar de Maclaurin colapsa y diverge catastróficamente si se evalúa con valores donde el valor absoluto de x es mayor o igual a 1.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

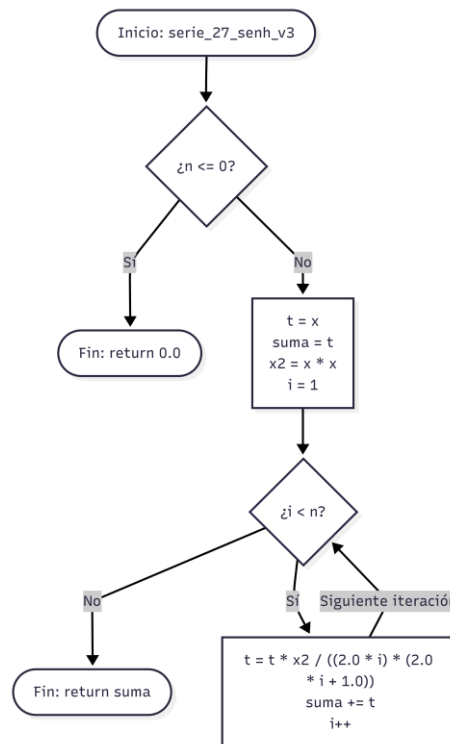


Mientras que la implementación de Nivel 1 intentaría forzar la serie estándar para cualquier valor, resultando en un desbordamiento de memoria o ciclos infinitos, la arquitectura de Nivel 3 implementa un enrutamiento dinámico de dominio. Como se observa en la bifurcación central del diagrama de flujo, el algoritmo audita la entrada: si $|x| < 1$, ejecuta la recurrencia $O(n)$ estándar. Si $|x| \geq 1$, el programa transmuta la entrada evaluando su inverso multiplicativo ($1/x$) y ajustando el resultado final mediante la identidad $\frac{\pi}{2} - suma$. Esta adaptación geométrica garantiza una convergencia rápida y una complejidad estricta de $O(n)$ sin importar la magnitud del dato ingresado, manteniendo la función completamente desacoplada y autosuficiente.

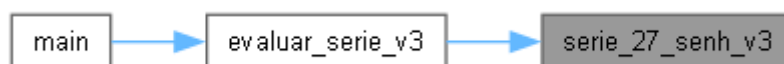
xxvii. **Serie 27: $\sinh(x)$**

El seno hiperbólico constituye una herramienta analítica fundamental en la formulación de modelos físicos, particularmente en la estática de cables suspendidos (catenarias) y en la resolución de ecuaciones diferenciales parciales asociadas a fenómenos de transferencia. Analíticamente, su expansión en serie de Maclaurin exhibe un radio de convergencia infinito, propiedad matemática que exige a la arquitectura del software de requerir validaciones restrictivas de dominio en sus condiciones de frontera.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

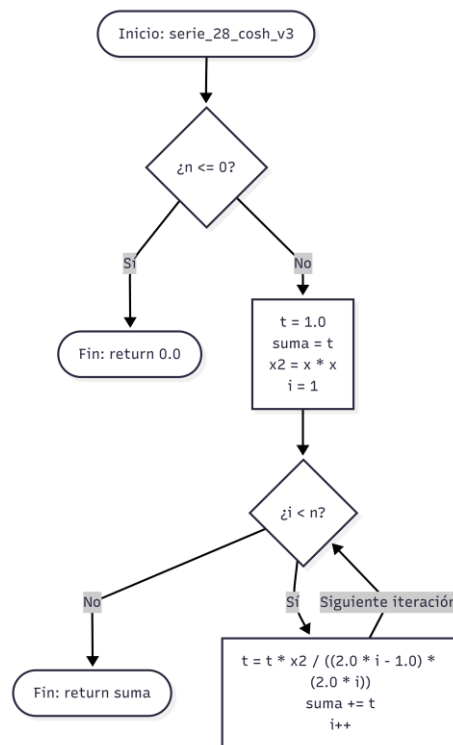


Desde la perspectiva del análisis de algoritmos, esta implementación consolida una cota asintótica de complejidad temporal estricta de $O(n)$. Al estar matemáticamente desprovista del término de alternancia de signo, el ciclo iterativo procesa una recurrencia algebraica monótona y directa. Adicionalmente, el grafo de dependencias generado por Doxygen ratifica una arquitectura monolítica completamente aislada. Esta independencia modular anula la sobrecarga en la pila de llamadas y favorece la predicción de saltos a nivel de hardware, garantizando un aprovechamiento óptimo de los ciclos de reloj del procesador.

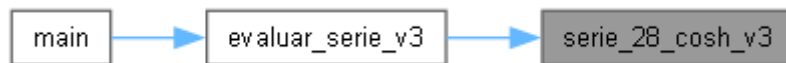
xxviii. Serie 28: $\cosh(x)$

El coseno hiperbólico es la contraparte analítica fundamental en el modelado bidimensional de catenarias y tensores físicos. Matemáticamente, su expansión de Maclaurin posee un radio de convergencia infinito, propiedad que exige al algoritmo de implementar guardias restrictivas de dominio, simplificando la lógica de entrada.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

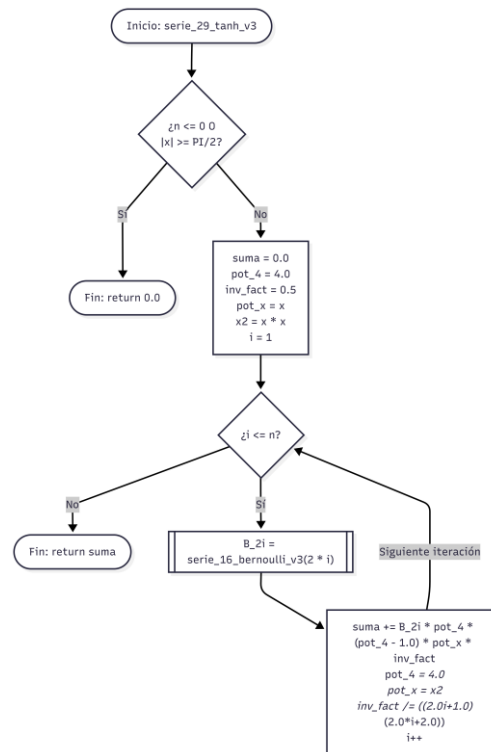


El diseño algorítmico consolida una cota asintótica temporal de $O(n)$ estricta. Al prescindir del coeficiente de alternancia de signo, la recurrencia iterativa es algebraicamente directa y monótona. Simultáneamente, el grafo de Doxygen evidencia una estructura monolítica aislada; esta carencia de dependencias externas suprime el overhead de saltos en memoria.

xxix. Serie 29: $\tanh(x)$

La tangente hiperbólica es una función indispensable en el diseño de redes neuronales artificiales (como función de activación no lineal) y en la propagación de ondas electromagnéticas. Su expansión de Maclaurin exhibe una alta complejidad teórica al depender intrínsecamente de la secuencia de Bernoulli, lo que restringe su radio de convergencia asintótica al dominio estricto de $\left(-\frac{\pi}{2}, \frac{\pi}{2}\right)$. Esta limitante impone la ejecución de una barrera de validación ineludible al inicio del flujo para evitar divergencias catastróficas.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

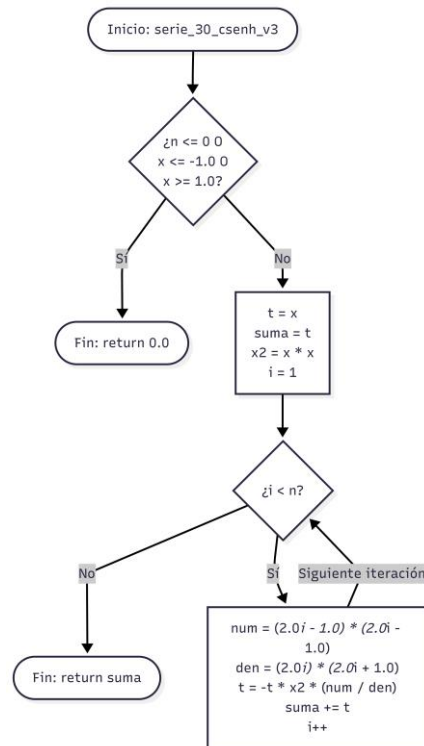


Como ilustra el grafo de dependencias de Doxygen, el bucle principal concentra los ciclos de reloj exclusivamente en la actualización asintótica de estados algebraicos, mientras delega la carga analítica crítica (la evaluación de coeficientes) a la subrutina especializada de Bernoulli.

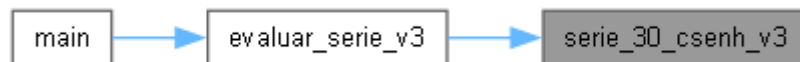
xxx. Serie 30: $\operatorname{arcsenh}(x)$

El arcoseno hiperbólico es una función inversa trascendental utilizada para linealizar crecimientos exponenciales y resolver integrales por sustitución en ingeniería física. Su expansión en serie de Maclaurin posee un radio de convergencia acotado al intervalo $[-1, 1]$, exigiendo una validación de dominio rigurosa para prevenir divergencias numéricas y asegurar la estabilidad del algoritmo bajo condiciones de frontera críticas.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

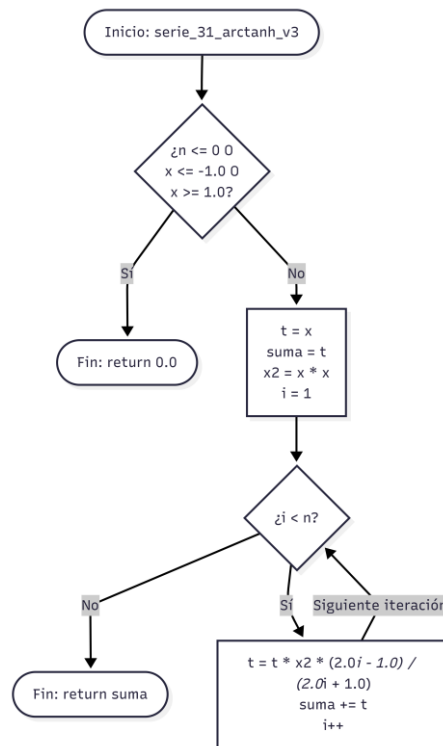


A diferencia del Nivel 1, el flujo iterativo actualiza el término de la serie multiplicando por factores fraccionarios precalculados, lo que elimina el uso de funciones trascendentales externas dentro del ciclo. Como indica el grafo de Doxygen, el diseño es monolítico y desacoplado, minimizando el costo de conmutación de contexto y maximizando el rendimiento por ciclo de instrucción.

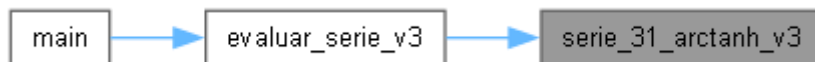
xxxi. Serie 31: $\text{arctanh}(x)$

La arcotangente hiperbólica es fundamental en la relatividad especial (para el cálculo de rapidez) y en el procesamiento estadístico de datos mediante la transformación de Fisher. Analíticamente, su serie de Maclaurin es válida únicamente en el intervalo abierto $(-1, 1)$; por ello, el algoritmo implementa una condicional de guarda estricta que inhabilita el procesamiento de valores fuera del radio de convergencia, garantizando la integridad numérica del sistema.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

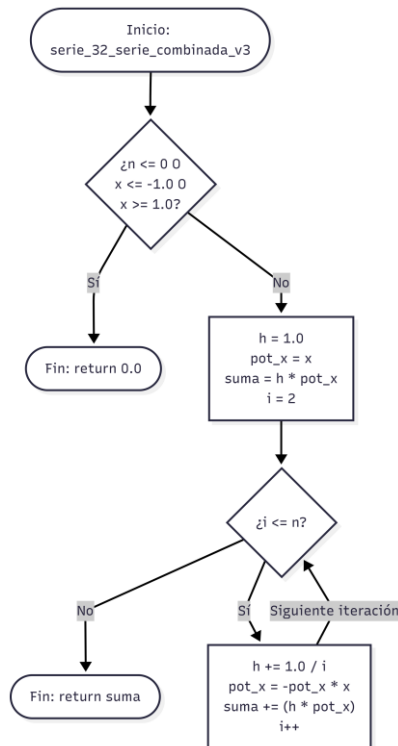


A diferencia de la arcotangente circular, esta versión hiperbólica simplifica la carga computacional al eliminar la alternancia de signos en el acumulador. El grafo de Doxygen confirma una estructura de baja acoplamiento y alta cohesión, lo que reduce la latencia en el flujo de instrucciones y optimiza el uso de registros del procesador para una convergencia acelerada.

xxxii. **Serie 32:** $\frac{\ln(1+x)}{1+x}$

La serie combinada es una función trascendental compleja que surge con frecuencia en el análisis de algoritmos y la teoría de probabilidades. Su expansión de Maclaurin es notable por integrar números armónicos en sus coeficientes, lo que impone un radio de convergencia acotado al intervalo $(-1, 1]$. El algoritmo gestiona esta restricción mediante una validación previa para asegurar la estabilidad asintótica del cálculo.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen

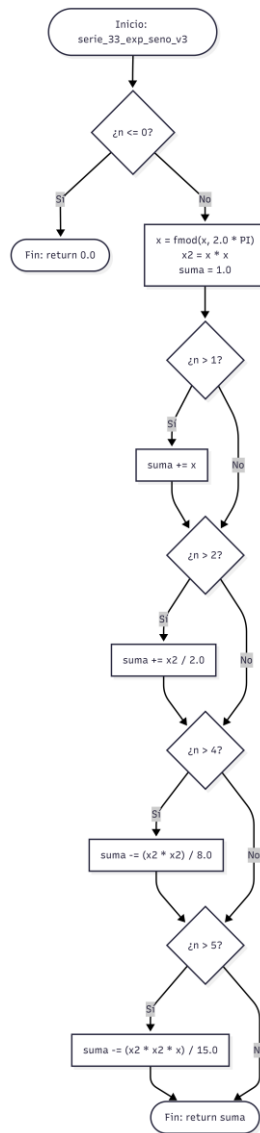


En lugar de calcular cada término de forma independiente, el algoritmo deduce el número armónico actual $\left(H_i = H_{i-1} + \frac{1}{i}\right)$ y la potencia alternante en un solo paso iterativo. El grafo de Doxygen confirma una estructura monolítica de alta eficiencia que minimiza la sobrecarga de memoria, permitiendo una convergencia precisa sin recurrir a funciones costosas de la librería math.h.

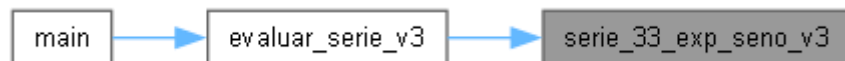
xxxiii. Serie 33: $e^{\text{sen}(x)}$

Es fundamental en el análisis de señales periódicas moduladas y en la resolución de problemas de difracción en óptica física. A diferencia de las series de Taylor convencionales, esta función presenta una complejidad analítica superior al combinar una función exponencial con una oscilatoria. Para maximizar la estabilidad numérica, el algoritmo implementa una reducción de dominio mediante aritmética modular (2π) , normalizando cualquier entrada al primer ciclo trigonométrico antes de su evaluación.

Diagrama de Flujo del nivel 3



Grafo de llamadas de Doxygen



Al prescindir de estructuras iterativas, la función alcanza una complejidad temporal constante de $O(1)$ para un número de términos predefinido, optimizando drásticamente el flujo de instrucciones en el procesador. El grafo de dependencias ratifica un diseño de "ruta crítica" directa, donde cada condicional actúa como un filtro de precisión, garantizando una respuesta inmediata y determinista sin el costo computacional de la recursividad o el mantenimiento de ciclos.

6. Conclusiones y Buenas Práctica

El cierre final del proyecto. En este proyecto se implementaron diversas series matemáticas con el objetivo de analizar su convergencia y precisión numérica, así como optimizar su cálculo mediante diferentes enfoques de programación. A lo largo del desarrollo, se comprobó que la forma en que se implementa una serie influye directamente en su rendimiento y estabilidad.

En términos de rendimiento, se observó que los métodos del Nivel 1, aunque matemáticamente correctos, resultan computacionalmente ineficientes debido al uso de ciclos anidados y al recálculo constante de potencias y factoriales, alcanzando complejidades del orden $O(n^2)$. En contraste, las implementaciones del Nivel 3, basadas en relaciones de recurrencia, reducen la complejidad a $O(n)$, lo que permite ejecutar un mayor número de iteraciones en menor tiempo y con mejor aprovechamiento de los recursos del sistema.

Asimismo, se identificó que técnicas como la suma en orden inverso, el uso de series telescópicas y la eliminación de operaciones redundantes contribuyen significativamente a mejorar la precisión numérica, disminuyendo los errores de redondeo acumulados.

Desde el punto de vista de ingeniería de software, se aplicaron buenas prácticas como la programación modular, el uso de funciones auxiliares y la documentación mediante Doxygen, lo que facilitó la organización, comprensión y mantenimiento del código. Además, el uso de estructuras eficientes y el análisis del comportamiento numérico permitieron desarrollar soluciones más robustas y escalables.

En conclusión, este proyecto permitió comprender que no basta con obtener una solución correcta desde el punto de vista matemático, sino que también es fundamental considerar la eficiencia computacional y la estabilidad numérica, aspectos clave en el desarrollo de software científico y en aplicaciones reales.

7. Bibliografía

- [1] Wolfram Research, Inc. (n.d.). *Natural Logarithm of 2 -- from Wolfram MathWorld*.
<https://mathworld.wolfram.com/NaturalLogarithmof2.html>
- [2] *Leibniz Series, calculating $\pi/4$* – fp64lib.org. (n.d.). <https://fp64lib.org/examples/leibniz-series-calculating-pi-4/>
- [3] mjabaco. (2019, November 20). *1.38 Demostrar ecuación: $\sum (1/n^2) = \pi^2/6$* [Video]. YouTube. <https://www.youtube.com/watch?v=cct5Ci74xPA>
- [4] paganerius. (2020, November 22). *EL PROBLEMA DE BASILEA* [Video]. YouTube. <https://www.youtube.com/watch?v=mDibCfDzfIU>
- [5] What is Pi? | Number & Examples - Lesson | Study.com. (n.d.). study.com. <https://study.com/learn/lesson/what-is-pi-number-usage.html>
- [6] Avila, A. (2019). Significados, representaciones y lenguaje: las fracciones en tres generaciones de libros de texto para primaria. *Educación Matemática*, 31(2), 22–60.
<https://doi.org/10.24844/em3102.02>
- [7] Courant, R., & Robbins, H. (1996). *What is Mathematics?: Un clásico que explica la convergencia de series aplicadas a problemas lógicos*.
- [8] Stewart, J. (2015). *Calculus: Early Transcendentals*.
- [9] Felipeo Física & Matemática en TikTok. (n.d.). TikTok. <https://www.tiktok.com/@felipao.fima/video/7301866283325263109>
- [10] Zaborowska, Ł. (2024, October 30). *Calculadora de e elevado a un número (e^x)*. Omni Calculator. <https://www.omnicalculator.com/es/matematicas/calculadora-e-elevado-x>
- [11] Corless, R. M., et al. (1996). *On the Lambert W function*.
- [12] Abramowitz, M., & Stegun, I. A. (1964). *Handbook of Mathematical Functions*.
- [13] Zill, D. G., & Wright, W. S. (2011). *Calculus: Early Transcendentals*.